

SE Trainee MCQ Cracker

THE SOFTWARE ENGINEER TRAINEE
MCQ CRACKER — 5-DAY HIGH-YIELD HANDBOOK

The 20% of concepts that answer 80% of
fresher SE MCQ questions

Targeted for: WellDev · Selise · Brain Station 23 ·
Enosis · Therap · BJIT · Cefalo · TigerIT · DataSoft

Exam-focused · No filler · Answer keys included

The SE Trainee MCQ Cracker

A senior engineer's 5-day crash guide for the Bangladeshi trainee hiring exam.

How to use this handbook

This is **not a textbook**. It's a weapon tuned for one thing: **maximizing your MCQ score** on a trainee software engineer exam. Everything is filtered through the **Pareto Principle** — the ~20% of concepts that show up in ~80% of questions. Low-frequency theory is deliberately cut.

Every topic follows the same rhythm: - **Why it matters** (with notes on which BD companies test it hardest) - **Essential concepts** — dense, no padding - **Frequently confused concepts** — the traps interviewers love - **Common mistakes & memory tricks** - **Cheat sheet + top facts to memorize** - **MCQs with a full answer key** (attempt first, then check)

💡 **The one rule:** For every MCQ, **attempt it before reading the answer**. Passive reading feels productive but doesn't build recall. Cover the answer key, commit to a choice, *then* check. That single habit is worth 10–15 marks.

Your 5-Day Plan

Day	Morning	Afternoon	Evening
Day 1	OOP (Topic 1)	Data Structures (2)	Complexity (4)
Day 2	Algorithms (3)	SQL (5)	Mock Test 1
Day 3	DBMS (6)	Operating Systems (7)	Review weak areas
Day 4	Computer Networks (8)	REST APIs (9) + Git (10)	Mock Test 2
Day 5	Design Patterns (11) + System Design (12)	CSE Fundamentals (13) + Math/Logic (14)	Final Rapid Revision

 **If you have less time:** The five highest-yield topics for BD trainee exams are **OOP, SQL, Data Structures, Complexity, and OS**. Do those first. They alone typically cover 55–65% of the paper.

What Bangladeshi trainee exams actually look like

Based on the pattern at the target companies:

- **Format:** 40–70 MCQs, 60–90 minutes. Often an online platform (HackerRank, TestGorilla, or in-house).
- **Some companies add** 1–2 short coding problems after the MCQ round (arrays, strings, basic logic). This handbook keeps you MCQ-ready; brush up basic coding separately.
- **Heaviest topics:** OOP, SQL, output-prediction (C/C++/Java), data structures, complexity (Big-O), OS basics, aptitude/logic.
- **Company leanings:**
 - **Therap, Brain Station 23, Enosis, BJIT** → Java-heavy OOP + DS/Algo + SQL.
 - **WellDev, Selise** → .NET/C#, JavaScript/TypeScript, plus general CS + logic.
 - **Cefalo, TigerIT, DataSoft** → strong CS fundamentals + aptitude.
- **Negative marking:** Some do, some don't. Assume it *might* — so **eliminate options** rather than wild-guess when unsure (technique covered in the final section).

Table of Contents

1. Object-Oriented Programming (OOP)
2. Data Structures
3. Algorithms
4. Time & Space Complexity
5. SQL
6. Database Management Systems (DBMS)
7. Operating Systems
8. Computer Networks
9. REST APIs
10. Git & Version Control
11. Design Patterns

- 12. Basic System Design
 - 13. CSE Fundamentals (grab-bag of high-frequency facts)
 - 14. Basic Mathematics & Logical Reasoning
 - 15. Mock Tests (2 × full papers with answer keys)
 - 16. Final Rapid Revision (must-know facts, exam-day strategy)
-

Mindset for exam day: You don't need to know everything. You need to reliably bank the questions you *do* know and make smart guesses on the rest. This handbook makes the "know" bucket as large as possible. Let's begin.

Topic 1 — Object-Oriented Programming (OOP)

 **Why it matters:** OOP is the **#1 most-asked theory topic** in BD trainee exams. Brain Station 23, Enosis, Therap, BJIT, Cefalo are Java/C#-heavy — expect 8–15 OOP MCQs. WellDev/Selise test the concepts plus JS/TS twists. OOP + SQL alone can be 35–40% of the paper.

Essential concepts (the 20%)

Class = blueprint; **Object** = instance. `Car myCar = new Car();`

The 4 Pillars — “A PIE”:

A – Abstraction → hide complexity, show essentials (WHAT not HOW)
P – Polymorphism → one name, many forms (overloading + overriding)
I – Inheritance → child reuses/extends parent (“is-a”)
E – Encapsulation → bundle data + methods; hide data (private + getters/setters)

Abstraction vs Encapsulation: Encapsulation hides **data** (private fields). Abstraction hides **implementation complexity** (abstract/interface). Steering wheel = abstraction; sealed engine = encapsulation.

Polymorphism — two forms:

```

      POLYMORPHISM
      /           \
Compile-time      Runtime
(Static)          (Dynamic)
OVERLOADING      OVERRIDING
same name, diff params  same signature, parent→child
```

Inheritance types: Single (A→B), Multilevel (A→B→C), Hierarchical (A→B, A→C), Multiple (via **interfaces only** in Java/C#; C++ allows class multiple inheritance).

Access modifiers (Java), least→most restrictive: `public` → `protected` (package + subclasses) → `default` (package) → `private` (class only).

Relationships:

Association (uses) < Aggregation (weak “has-a”) < Composition (strong “owns-a”)
Dept ◇— Professor (profs survive) House ◆— Room (rooms die with house)

Frequently confused concepts

A	B	Difference
Overloading	Overriding	same name/diff params/ compile-time vs same signature/ runtime
Abstract class	Interface	has state+constructor+concrete methods, single-inherit vs pure contract, multi-implement
Overriding	Hiding	instance methods (runtime, object type) vs static methods/fields (compile-time, reference type)
Cohesion	Coupling	want HIGH (focused class) vs want LOW (few dependencies)
Deep copy	Shallow copy	copies nested objects recursively vs copies references (shared nested object)
Abstraction	Encapsulation	detail hiding (design) vs data hiding (implementation)

Common mistakes & memory tricks

- ✗ "Overloading is runtime" → it's **compile-time**. **OverrLoading** = **compilE-time**.
- ✗ "Java supports multiple class inheritance" → **No** (Diamond Problem); interfaces only.
- ✗ "Abstract class has no concrete methods" → it **can** have them, plus a constructor.
- ✗ "static methods can be overridden" → they're **hidden**.
- ✗ Constructors are static/final/inherited → **none** of these; they **can** be overloaded.
- 🧠 **is-a = inheritance, has-a = composition**. Bird is-a Animal → **abstract**; Bird can Fly → **interface**.
- 🧠 Good design = **High Cohesion, Low Coupling (HCLC)**.

Output-prediction traps (study the reasoning)

```
// Trap 1: overriding (instance) → object type
class A { void show(){ System.out.print("A"); } }
class B extends A { void show(){ System.out.print("B"); } }
A o = new B(); o.show();           // → B

// Trap 2: hiding (static) → reference type
class A { static void show(){ System.out.print("A"); } }
class B extends A { static void show(){ System.out.print("B"); } }
A o = new B(); o.show();           // → A ← the classic trick!
```

Syntax facts: interface can't have a constructor; abstract class can't be instantiated (but has a constructor); **final** method can't be overridden; **final** class can't be extended; `class B extends A implements X, Y` is valid.

Cheat sheet

```
4 PILLARS (A PIE): Abstraction, Polymorphism, Inheritance, Encapsulation
Overloading = compile-time, diff params | Overriding = runtime, same signature
Abstract class: state+ctor+single | Interface: contract+multiple(+default methods 8+)
Java: NO multiple class inheritance (Diamond) → interfaces
Access: private < default < protected < public
STATIC → hidden (reference type) | INSTANCE → overridden (object type)
Composition(owns-a) > Aggregation(has-a) > Association(uses)
Design goal: HIGH cohesion, LOW coupling
Constructor: not static/final/inherited/overridden; CAN be overloaded
```

★ Top facts

A PIE · overloading=compile · overriding=runtime · no multiple class inheritance in Java · interfaces enable multiple inheritance · abstract class has constructor · interface has none · static=hidden · final method not overridable · encapsulation=data hiding · abstraction=detail hiding · composition=strong owns-a · HCLC · super=parent, this=current.

MCQs — attempt, then check the key at the end

Beginner (1–15) 1. OOP stands for? a) Object Ordered Programming b) Object Oriented Programming c) Ordered Object Protocol d) Operational Object Processing 2. NOT a pillar of OOP? a) Encapsulation b) Polymorphism c) Compilation d) Inheritance 3. A class is a: a) Memory location b) Blueprint for objects c) Running instance d) Function 4. Encapsulation is about: a) Multiple forms b) Bundling data + hiding via access control c) Code reuse d) Hiding complexity 5. Java keyword for inheritance? a) implement b) inherits c) extends d) super 6. “One name, many forms” = a) Inheritance b) Encapsulation c) Polymorphism d) Abstraction 7. Overloading is resolved at: a) Runtime b) Compile-time c) Link-time d) Load-time 8. Overriding is resolved at: a) Runtime b) Compile-time c) Design-time d) Never 9. Inheritance represents which relationship? a) has-a b) uses-a c) is-a d) owns-a 10. Most restrictive Java access modifier? a) public b) protected c) default d) private 11. Abstraction hides: a) Data b) Objects c) Implementation complexity d) Class name 12. Multiple inheritance of classes in Java is: a) Allowed b) Not allowed c) Allowed with final d) Allowed with static 13. Multiple inheritance in Java is enabled via: a) Abstract classes b) Interfaces c) Packages d) Constructors 14. Getters/setters implement: a) Inheritance b) Polymorphism c) Encapsulation d) Abstraction 15. Which cannot be instantiated? a) Concrete class b) Abstract class c) Final class d) Static nested class

Intermediate (16–25) 16. A `o = new B();` where B overrides instance method `f()`. `o.f()` runs: a) A's f b) B's f c) compile error d) undefined 17. If `f()` were `static` in the above, `o.f()` runs: a) A's f b) B's f c) error d) both 18. Which is TRUE? a) Interface can have a constructor b) Abstract class can have a constructor c) Constructors are inherited d) Static methods are overridden 19. Composition implies: a) weak has-a b) strong owns-a (part dies with whole) c) is-a d) uses-a 20. Good design favors: a) high coupling b) low cohesion c) high cohesion, low coupling d) high coupling, high cohesion 21. Overloading requires: a) same name, different parameters b) different name, same params c) same name, same params d) different return type only 22. A `final` method: a) can be overridden b) cannot be overridden c) cannot be called d) must be static 23. Pre-Java 8 interface contains: a) only concrete methods b) only abstract methods + constants c)

constructors d) private fields 24. Diamond problem is solved in Java by: a) multiple class inheritance b) interfaces c) final classes d) static methods 25. Which best shows encapsulation? a) `public int x;` b) `private int x;` + getters/setters c) many-parameter method d) two same-name methods

Difficult (26–30) 26. In Java, `List l = new ArrayList();` demonstrates: a) overloading b) upcasting/polymorphism via reference type c) encapsulation d) static binding of the object 27. Runtime polymorphism in Java uses: a) static binding b) dynamic method dispatch (vtable) c) compile-time resolution d) overloading 28. Which statement is FALSE? a) An abstract class can have all concrete methods b) An interface method can have a body in Java 8+ c) A private method can be overridden d) Constructors can be overloaded 29. Shallow copy of an object with a nested list means: a) nested list is duplicated b) both objects share the same nested list reference c) compile error d) nested list becomes null 30. `super()` in a subclass constructor: a) calls the subclass constructor b) calls the parent constructor c) is illegal d) creates a new object

✅ Answer Key — Topic 1

1-b · 2-c · 3-b · 4-b · 5-c · 6-c · 7-b · 8-a · 9-c · 10-d · 11-c · 12-b · 13-b · 14-c · 15-b · 16-b · 17-a · 18-b · 19-b · 20-c · 21-a · 22-b · 23-b · 24-b · 25-b · 26-b · 27-b · 28-c · 29-b · 30-b

Key explanations: **8** Overloading = compile-time (static binding). **17** Static methods are *hidden*, resolved by reference type A. **18** Only abstract classes have constructors (called via `super()`); interfaces don't; constructors aren't inherited; static methods aren't overridden. **26** Reference type `List`, object `ArrayList` — upcasting enables polymorphism. **27** Runtime polymorphism = dynamic dispatch via the virtual method table. **28** FALSE because a `private` method is not visible to subclasses, so it **cannot** be overridden (only hidden/redefined independently). **29** Shallow copy shares the nested reference — a top trap.

Topic 2 — Data Structures

 **Why it matters:** Data structures + complexity are the backbone of every trainee exam. Enosis, Therap, BJIT, TigerIT test array/linked-list/stack/queue/tree/hash properties and their operation costs constantly. Most questions are “**which operation is $O(?)$** ” or “**which DS fits this use case.**”

Essential concepts

Array — contiguous memory, fixed size, **$O(1)$ index access**, but insert/delete in middle is **$O(n)$** (shifting).

Linked List — nodes with pointers. Insert/delete at known position **$O(1)$** , but access/search **$O(n)$** (no indexing). Singly, Doubly, Circular.

```
Array:  [10][20][30][40]      index access =  $O(1)$ 
LinkedList: 10→20→30→40→null  access =  $O(n)$ , insert head =  $O(1)$ 
```

Stack — **LIFO** (Last In First Out). Operations: **push**, **pop**, **peek** — all **$O(1)$** . Uses: function call stack, undo, expression evaluation, backtracking, DFS.

Queue — **FIFO** (First In First Out). **enqueue** (rear), **dequeue** (front) — **$O(1)$** . Uses: scheduling, BFS, buffers. Variants: Circular queue, Deque, Priority queue.

```
Stack (LIFO):  push→ [ 3 ] ←pop from top
                [ 2 ]
                [ 1 ]
Queue (FIFO):  enqueue→ [1][2][3] →dequeue (front=1)
```

Hash Table / HashMap — key→value via hash function. Average **$O(1)$** insert/search/delete; **worst $O(n)$** (collisions). Collisions handled by **chaining** (linked lists) or **open addressing** (probing).

Tree — hierarchical; each node has children. **Binary tree**: ≤ 2 children. **BST (Binary Search Tree)**: left < node < right → search/insert/delete **$O(\log n)$** if balanced, **$O(n)$** if skewed.

Balanced trees (AVL, Red-Black) guarantee **$O(\log n)$** . A **skewed BST degenerates to a linked list ($O(n)$)** — common trap.

Heap — complete binary tree. **Min-heap**: parent $\leq$ children (root = min). **Max-heap**: parent $\geq$ children (root = max). Insert/delete **$O(\log n)$** , get-min/max **$O(1)$** . Basis of **priority queue** and **heap sort**.

Graph — vertices + edges. Represented by **adjacency matrix** ($O(V^2)$ space, $O(1)$ edge check) or **adjacency list** ($O(V+E)$ space, better for sparse graphs).

Tree traversals:



Inorder (L,Root,R): D B E A C
 Preorder (Root,L,R): A B D E C
 Postorder(L,R,Root): D E B C A
 Level-order (BFS): A B C D E

🗨 **Inorder of a BST = sorted order.** This is a favorite MCQ.

Frequently confused concepts

A	B	Difference
Array	Linked list	O(1) access/fixed size vs O(n) access/dynamic size
Stack	Queue	LIFO vs FIFO
BST search	Balanced BST	O(n) worst (skewed) vs guaranteed O(log n)
Min-heap	BST	only parent-child order (root=min) vs full left<node<right ordering
Adjacency matrix	Adjacency list	O(V ²) dense vs O(V+E) sparse
DFS	BFS	stack/recursion, goes deep vs queue, goes level-by-level

Which DS to use? (common MCQ)

Need	Use
Fast index access	Array
Frequent insert/delete at ends	Linked list / Deque
LIFO / undo / DFS / call stack	Stack
FIFO / scheduling / BFS	Queue
Fast key lookup	Hash table
Sorted data + fast search	Balanced BST
Always get min/max quickly	Heap / Priority queue
Relationships/networks	Graph

Operation complexity table (MEMORIZE — this is exam gold)

Structure	Access	Search	Insert	Delete
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Sorted array	$O(1)$	$O(\log n)$	$O(n)$	$O(n)$
Linked list	$O(n)$	$O(n)$	$O(1)^*$	$O(1)^*$
Stack/Queue	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Hash table	—	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg
BST (balanced)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
BST (skewed)	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Heap	—	$O(n)$	$O(\log n)$	$O(\log n)$

*at a known position / at the head.

Common mistakes & tricks

- ✗ "Array access is $O(n)$ " → it's **$O(1)$** (direct index).
- ✗ "Linked list access is $O(1)$ " → **$O(n)$** (must traverse).
- ✗ "Hash table is always $O(1)$ " → **average** $O(1)$, **worst** $O(n)$.
- ✗ "BST is always $O(\log n)$ " → only if **balanced**; skewed = $O(n)$.
- 🧠 **Stack = LIFO = plates; Queue = FIFO = line at a shop.**
- 🧠 **Inorder BST = sorted. BFS = queue, DFS = stack.**
- 🧠 A **complete binary tree** with n nodes has height $\lfloor \log_2 n \rfloor$.

Cheat sheet

Array: $O(1)$ access, $O(n)$ insert/delete | LinkedList: $O(n)$ access, $O(1)$ insert@head
 Stack LIFO (push/pop/peek $O(1)$) | Queue FIFO (enqueue/dequeue $O(1)$)
 Hash: $O(1)$ avg / $O(n)$ worst (collisions → chaining/open addressing)
 BST balanced $O(\log n)$, skewed $O(n)$ | Heap: root=min/max $O(1)$, insert $O(\log n)$
 Traversals: In(LNR)=sorted for BST, Pre(NLR), Post(LRN), Level=BFS
 BFS→queue, DFS→stack/recursion | Adjacency: matrix $O(V^2)$, list $O(V+E)$

MCQs — attempt, then check key

Beginner (1–15) 1. Which gives $O(1)$ access by index? a) Linked list b) Array c) Stack d) Queue 2. Stack follows: a) FIFO b) LIFO c) random d) priority 3. Queue follows: a) LIFO b) FIFO c) random d) sorted 4. Push and pop belong to: a) Queue b) Stack c) Tree d) Graph 5. Enqueue/dequeue belong to: a) Stack b) Queue c) Heap d) Array 6. Linked list access time is: a) $O(1)$ b) $O(\log n)$ c) $O(n)$ d) $O(n^2)$ 7. In a min-heap, the root is the: a) maximum b) minimum c) median d) last inserted 8. Inorder traversal of a BST gives: a) reverse order b) sorted order c) random d) level order 9. BFS uses which structure? a) Stack b) Queue c) Heap d) Array 10. DFS typically uses: a) Queue b) Stack/recursion c) Heap d) Hash 11. A binary tree node has at most: a) 1 child b) 2 children c) 3 children d) unlimited 12. Hash table average lookup is: a) $O(n)$ b) $O(\log n)$ c) $O(1)$ d) $O(n^2)$ 13.

Which is LIFO? a) Queue b) Stack c) Linked list d) Array 14. A complete graph with n vertices, adjacency matrix space is: a) $O(n)$ b) $O(n \log n)$ c) $O(n^2)$ d) $O(1)$ 15. Which DS is best for “undo” functionality? a) Queue b) Stack c) Heap d) Graph

Intermediate (16–25) 16. Worst-case search in a hash table is: a) $O(1)$ b) $O(\log n)$ c) $O(n)$ d) $O(n^2)$ 17. A skewed BST search is: a) $O(1)$ b) $O(\log n)$ c) $O(n)$ d) $O(n^2)$ 18. Preorder of tree (root A, left B, right C; B has children D,E): a) DBEAC b) ABDEC c) DEBCA d) ABCDE 19. Which handles hash collisions? a) recursion b) chaining / open addressing c) balancing d) heapifying 20. Deleting from the middle of a singly linked list (node given, with head traversal) is: a) $O(1)$ b) $O(\log n)$ c) $O(n)$ d) $O(n^2)$ 21. A priority queue is typically implemented with a: a) stack b) heap c) hash d) array only 22. Which traversal is BFS? a) inorder b) preorder c) postorder d) level-order 23. Adjacency list space complexity: a) $O(V)$ b) $O(E)$ c) $O(V+E)$ d) $O(V^2)$ 24. In a max-heap, which is guaranteed? a) root is smallest b) parent $\geq$ children c) it is sorted d) left < right 25. Binary search requires the array to be: a) unsorted b) sorted c) a linked list d) a heap

Difficult (26–30) 26. Height of a balanced binary tree with n nodes: a) $O(n)$ b) $O(\log n)$ c) $O(1)$ d) $O(n \log n)$ 27. A queue implemented with two stacks has amortized dequeue: a) $O(n)$ b) $O(1)$ c) $O(\log n)$ d) $O(n^2)$ 28. Which structure gives $O(1)$ insert AND $O(1)$ delete at both ends? a) stack b) singly linked list c) deque (doubly linked) d) array 29. Number of nodes in a perfect binary tree of height h : a) 2^h b) $2^{h+1} - 1$ c) h^2 d) $2h$ 30. In-place heap sort time complexity: a) $O(n)$ b) $O(n \log n)$ c) $O(n^2)$ d) $O(\log n)$

✅ Answer Key — Topic 2

1-b · 2-b · 3-b · 4-b · 5-b · 6-c · 7-b · 8-b · 9-b · 10-b · 11-b · 12-c · 13-b · 14-c · 15-b · 16-c · 17-c · 18-b · 19-b · 20-c · 21-b · 22-d · 23-c · 24-b · 25-b · 26-b · 27-b · 28-c · 29-b · 30-b

Key explanations: **17** Skewed BST behaves like a linked list $\rightarrow O(n)$. **18** Preorder = Root, Left, Right = A B D E C. **27** Amortized $O(1)$ — each element is moved between stacks at most twice. **29** Perfect binary tree of height h has $2^{h+1} - 1$ nodes. **30** Heap sort is always $O(n \log n)$.

Topic 3 — Algorithms

 **Why it matters:** Sorting/searching complexities and “which algorithm” questions are guaranteed. BJIT, Enosis, TigerIT love **sorting comparisons**, **binary search**, and **recursion output**. You rarely code these in the MCQ — you must know their **time/space costs, stability, and when to use each**.

Searching

Linear search — check each element. $O(n)$. Works on unsorted data.

Binary search — repeatedly halve a **sorted** array. $O(\log n)$. Requires sorted input.

Find 7 in [1,3,5,7,9,11]: mid=5<7 → right half → mid=9>7 → left → 7 ✓
Each step halves the search space → $\log_2 n$ steps

Sorting — the master table (MEMORIZE)

Algorithm	Best	Average	Worst	Space	Stable?
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	✗
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	✓
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	✗
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	✗
Counting	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	✓

Key facts interviewers test: - **Quicksort worst case = $O(n^2)$** (already-sorted array with bad pivot). Average $O(n \log n)$. Usually **fastest in practice**. - **Merge sort is always $O(n \log n)$** but needs **$O(n)$ extra space**. **Stable**. Good for linked lists / external sorting. - **Heap sort = $O(n \log n)$, in-place $O(1)$ space**, but not stable. - **Insertion sort is best for nearly-sorted / small data** — $O(n)$ best case. - **Stable sort** preserves relative order of equal elements. Bubble, Insertion, Merge, Counting = stable. - **Comparison-based sorts can't beat $O(n \log n)$** . Counting/Radix/Bucket beat it by *not* comparing ($O(n+k)$) but need special conditions.

Recursion

A function calling itself with a **base case** (stops recursion) + **recursive case**.

```
factorial(n): if n==0 return 1; else return n * factorial(n-1)
factorial(3) = 3*factorial(2)=3*2*factorial(1)=3*2*1*factorial(0)=3*2*1*1=6
```

 **No base case** → **infinite recursion** → **stack overflow**. Each call adds a stack frame → recursion uses **$O(\text{depth})$ space**.

Fibonacci naive recursion = $O(2^n)$ (exponential, recomputes). With **memoization/DP = $O(n)$** . Classic MCQ.

Algorithmic paradigms (know the one-liners)

Paradigm	Idea	Examples
Divide & Conquer	split → solve → combine	Merge sort, Quick sort, Binary search
Greedy	pick locally best choice	Dijkstra, Prim, Kruskal, Huffman, Activity selection
Dynamic Programming	overlapping subproblems + optimal substructure; store results	Fibonacci, Knapsack, LCS, edit distance
Backtracking	try, and undo if it fails	N-Queens, Sudoku, permutations

🟡 **Greedy vs DP:** Greedy makes one irreversible choice at each step (fast, not always optimal). DP explores all subproblems and stores answers (optimal, more memory).

Graph algorithms (high-yield names)

- **BFS** (queue) — shortest path in **unweighted** graph. $O(V+E)$.
- **DFS** (stack/recursion) — cycle detection, topological sort, connectivity. $O(V+E)$.
- **Dijkstra** — shortest path, **weighted, non-negative** edges. Greedy.
- **Bellman-Ford** — shortest path, handles **negative** edges. $O(VE)$.
- **Kruskal / Prim** — Minimum Spanning Tree (MST). Greedy.

Common mistakes & tricks

- ❌ "Quicksort is always $O(n \log n)$ " → **worst $O(n^2)$** .
- ❌ "Merge sort is in-place" → needs **$O(n)$** extra space.
- ❌ "Binary search works on any array" → must be **sorted**.
- ❌ "Fibonacci recursion is $O(n)$ " → naive is **$O(2^n)$** ; DP makes it $O(n)$.
- 🟡 **Stable sorts: B-I-M-C** (Bubble, Insertion, Merge, Counting).
- 🟡 **Divide & Conquer trio: Merge, Quick, Binary search.**
- 🟡 **BFS = shortest path in unweighted; Dijkstra = weighted (non-negative).**

Cheat sheet

```
Linear search  $O(n)$  unsorted | Binary search  $O(\log n)$  SORTED
Merge: always  $O(n \log n)$ ,  $O(n)$  space, STABLE
Quick: avg  $O(n \log n)$ , worst  $O(n^2)$ , in-place-ish, fastest in practice
Heap:  $O(n \log n)$ ,  $O(1)$  space | Insertion:  $O(n)$  best (nearly sorted)
Comparison sorts  $\geq O(n \log n)$  | Counting/Radix  $O(n+k)$  non-comparison
Recursion needs base case; naive Fibonacci  $O(2^n)$ , DP  $O(n)$ 
Greedy(1 choice) vs DP(all subproblems+store) vs Backtracking(try+undo)
BFS=queue/unweighted shortest | DFS=stack | Dijkstra=weighted non-neg
```

MCQs — attempt, then check key

Beginner (1–15) 1. Binary search requires the array to be: a) unsorted b) sorted c) reversed d) circular 2. Time complexity of linear search: a) $O(1)$ b) $O(\log n)$ c) $O(n)$ d) $O(n^2)$ 3. Time complexity of binary search: a) $O(1)$ b) $O(\log n)$ c) $O(n)$ d) $O(n^2)$ 4. Merge sort worst case: a) $O(n)$ b) $O(n \log n)$ c) $O(n^2)$ d) $O(2^n)$ 5. Quicksort worst case: a) $O(n \log n)$ b) $O(n^2)$ c) $O(\log n)$ d) $O(n)$ 6. Which sort is stable? a) Selection b) Quick c) Merge d) Heap 7. Recursion must have a: a) loop b) base case c) global variable d) pointer 8. Merge sort extra space: a) $O(1)$ b) $O(\log n)$ c) $O(n)$ d) $O(n^2)$ 9. Which uses divide and conquer? a) Bubble sort b) Merge sort c) Linear search d) Counting sort 10. BFS uses a: a) stack b) queue c) heap d) tree 11. Naive recursive Fibonacci complexity: a) $O(n)$ b) $O(n^2)$ c) $O(2^n)$ d) $O(\log n)$ 12. Best case of insertion sort (nearly sorted): a) $O(1)$ b) $O(n)$ c) $O(n \log n)$ d) $O(n^2)$ 13. Dijkstra's algorithm finds: a) MST b) shortest path (non-negative weights) c) cycles d) sorting 14. Which paradigm stores subproblem results? a) Greedy b) Dynamic Programming c) Backtracking d) Brute force 15. Heap sort space complexity: a) $O(1)$ b) $O(n)$ c) $O(\log n)$ d) $O(n^2)$

Intermediate (16–25) 16. Comparison-based sorting lower bound: a) $O(n)$ b) $O(n \log n)$ c) $O(\log n)$ d) $O(n^2)$ 17. Which sort is best for a linked list? a) Quick b) Merge c) Heap d) Selection 18. Counting sort time complexity: a) $O(n \log n)$ b) $O(n+k)$ c) $O(n^2)$ d) $O(\log n)$ 19. DFS is typically implemented with: a) queue b) stack/recursion c) heap d) hash 20. Which finds shortest path with negative edges? a) Dijkstra b) Bellman-Ford c) BFS d) Prim 21. Greedy vs DP: greedy makes: a) all choices b) one irreversible local choice c) random choices d) no choices 22. Kruskal's algorithm builds: a) shortest path b) MST c) topological order d) hash table 23. Which is NOT divide and conquer? a) Merge sort b) Quick sort c) Binary search d) Bubble sort 24. Selection sort is stable? a) yes b) no c) sometimes d) only for integers 25. Which sort is generally fastest in practice for arrays? a) Bubble b) Quick c) Selection d) Insertion

Difficult (26–30) 26. Master theorem: $T(n)=2T(n/2)+O(n)$ solves to: a) $O(n)$ b) $O(n \log n)$ c) $O(n^2)$ d) $O(\log n)$ 27. Number of comparisons in binary search over n elements (worst): a) n b) $\lceil \log_2 n \rceil$ c) $n/2$ d) n^2 28. Topological sort applies to: a) any graph b) a DAG (directed acyclic graph) c) undirected graph d) weighted graph only 29. Space complexity of recursion of depth d : a) $O(1)$ b) $O(d)$ c) $O(2^d)$ d) $O(\log d)$ 30. 0/1 Knapsack is solved optimally by: a) greedy b) dynamic programming c) binary search d) DFS only

✓ Answer Key — Topic 3

1-b · 2-c · 3-b · 4-b · 5-b · 6-c · 7-b · 8-c · 9-b · 10-b · 11-c · 12-b · 13-b · 14-b · 15-a · 16-b · 17-b · 18-b · 19-b · 20-b · 21-b · 22-b · 23-d · 24-b · 25-b · 26-b · 27-b · 28-b · 29-b · 30-b

Key explanations: 8 Merge sort needs $O(n)$ auxiliary array. 11 Naive Fibonacci recomputes $\rightarrow O(2^n)$. 17 Merge sort suits linked lists (no random access needed, stable). 26 Master theorem case $\rightarrow O(n \log n)$. 28 Topological sort only exists for a DAG. 30 Fractional knapsack = greedy, but **0/1 knapsack = DP**.

Topic 4 — Time & Space Complexity (Big-O)

 **Why it matters:** Almost **every** BD trainee paper has 3–6 pure Big-O questions (“what’s the complexity of this loop?”). This is the easiest topic to master for guaranteed marks. WellDev, Selise, Enosis all include loop-analysis questions.

The core idea

Big-O describes how runtime/space grows as input size **n** grows — the **worst-case upper bound**, ignoring constants and lower-order terms. $3n^2 + 5n + 2 \rightarrow O(n^2)$.

Notations: **O** = upper bound (worst), **Ω** = lower bound (best), **Θ** = tight bound (both).

Growth-rate ranking (memorize the order)

$O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$
fast $\xrightarrow{\hspace{10em}}$ slow
constant log sqrt linear linearithmic quadratic cubic expo factorial

Complexity	Name	Example
$O(1)$	Constant	array index, hash lookup (avg)
$O(\log n)$	Logarithmic	binary search, balanced BST op
$O(n)$	Linear	single loop over n
$O(n \log n)$	Linearithmic	merge/quick/heap sort
$O(n^2)$	Quadratic	nested loop, bubble sort
$O(2^n)$	Exponential	naive Fibonacci, subsets
$O(n!)$	Factorial	permutations, brute-force TSP

How to read loops (the whole skill)

```
for (i=0; i<n; i++) { ... }           //  $O(n)$     – one loop

for (i=0; i<n; i++)
  for (j=0; j<n; j++) { ... }         //  $O(n^2)$   – nested, both n

for (i=0; i<n; i++)
  for (j=0; j<m; j++) { ... }         //  $O(n*m)$ 

for (i=1; i<n; i=i*2) { ... }          //  $O(\log n)$  – i doubles each time

for (i=0; i<n; i++)
  for (j=i; j<n; j++) { ... }         //  $O(n^2)$   –  $n+(n-1)+\dots+1 = n(n+1)/2$ 

i=n; while(i>1){ i=i/2; }             //  $O(\log n)$  – halving
```

Rules: - **Multiply** for nested loops, **add** for sequential loops. - A loop that **multiplies/divides** the counter ($i*=2$, $i/=2$) → **$O(\log n)$** . - Drop constants and non-dominant terms: $O(2n) \rightarrow O(n)$, $O(n^2+n) \rightarrow O(n^2)$. - Two separate loops = $O(n)+O(n)=O(n)$, not $O(n^2)$.

Amortized vs worst-case

Amortized = average cost per operation over a sequence. E.g., dynamic array (`ArrayList`) `add()` is **amortized $O(1)$** even though an occasional resize is $O(n)$.

Space complexity

Counts **extra** memory used (not the input). Recursion of depth d = **$O(d)$** stack space. Merge sort = $O(n)$ space; in-place algorithms = $O(1)$.

Common mistakes & tricks

- ✗ "Two sequential loops = $O(n^2)$ " → **$O(n)$** (add, don't multiply).
- ✗ " $i *= 2$ loop is $O(n)$ " → **$O(\log n)$** .
- ✗ Confusing worst vs average (hash: avg $O(1)$, worst $O(n)$).
- ✗ Keeping constants: $O(5n)$ is just **$O(n)$** .
- 🟡 **Halving/doubling → log. Nested independent loops → multiply.**
- 🟡 `for(i..n) for(j=i..n)` = **triangular = $O(n^2)$** ($\frac{1}{2}n^2$ but drop the $\frac{1}{2}$).

Cheat sheet

```
Order:  $1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n!$ 
Single loop  $O(n)$  | Nested  $O(n^2)$  |  $i*=2$  or  $i/=2 \rightarrow O(\log n)$ 
Nested loops → MULTIPLY | Sequential loops → ADD
Drop constants + lower terms:  $3n^2+5n \rightarrow O(n^2)$ 
Binary search  $O(\log n)$  | Merge/Quick/Heap  $O(n \log n)$  | Bubble  $O(n^2)$ 
Recursion space =  $O(\text{depth})$  | Dynamic array add = amortized  $O(1)$ 
```

MCQs — attempt, then check key

Beginner (1–15) 1. Big-O of a single loop `for(i=0; i<n; i++)`: a) $O(1)$ b) $O(n)$ c) $O(n^2)$ d) $O(\log n)$ 2. Big-O of two nested loops each running n times: a) $O(n)$ b) $O(n \log n)$ c) $O(n^2)$ d) $O(2n)$ 3. `for(i=1; i<n; i*=2)` runs: a) $O(n)$ b) $O(\log n)$ c) $O(n^2)$ d) $O(1)$ 4. Fastest growth (slowest algorithm) among these: a) $O(n)$ b) $O(n^2)$ c) $O(2^n)$ d) $O(\log n)$ 5. $O(2n + 3)$ simplifies to: a) $O(2n)$ b) $O(n)$ c) $O(3)$ d) $O(2)$ 6. Array index access complexity: a) $O(1)$ b) $O(n)$ c) $O(\log n)$ d) $O(n^2)$ 7. Binary search complexity: a) $O(n)$ b) $O(\log n)$ c) $O(1)$ d) $O(n^2)$ 8. Which is the tight bound notation? a) O b) Ω c) Θ d) λ 9. Bubble sort worst case: a) $O(n)$ b) $O(n \log n)$ c) $O(n^2)$ d) $O(1)$ 10. $O(n^2 + n)$ simplifies to: a) $O(n)$ b) $O(n^2)$ c) $O(n^3)$ d) $O(2n^2)$ 11. Which is fastest? a) $O(n)$ b) $O(1)$ c) $O(\log n)$ d) $O(n^2)$ 12. Two separate (sequential) loops over n : a) $O(n^2)$ b) $O(n)$ c) $O(\log n)$ d) $O(1)$ 13. Recursion depth d uses space: a) $O(1)$ b) $O(d)$ c) $O(2^d)$ d) $O(\log d)$ 14. Merge sort time: a) $O(n)$ b) $O(n \log n)$ c) $O(n^2)$ d) $O(2^n)$ 15. Hash table average lookup: a) $O(1)$ b) $O(n)$ c) $O(\log n)$ d) $O(n^2)$

Intermediate (16–25) 16. `for(i=0; i<n; i++) for(j=i; j<n; j++)` is: a) $O(n)$ b) $O(n \log n)$ c) $O(n^2)$ d) $O(\log n)$ 17. `while(n>1) n=n/2;` complexity: a) $O(n)$ b) $O(\log n)$ c) $O(1)$ d) $O(n^2)$ 18. Amortized complexity of `ArrayList.add()`: a) $O(n)$ b) $O(1)$ c) $O(\log n)$ d) $O(n^2)$ 19. $O(n) + O(n^2) + O(\log n)$ overall: a) $O(n)$ b) $O(\log n)$ c) $O(n^2)$ d) $O(n^3)$ 20. `for(i=0; i<n; i++) for(j=0; j<m; j++)`: a) $O(n)$ b) $O(n+m)$ c) $O(n \cdot m)$ d) $O(n^2)$ 21. Two nested loops but inner runs constant 100 times: a) $O(n)$ b) $O(100n) = O(n)$ c) $O(n^2)$ d) both a and b 22. Best case of insertion sort: a) $O(1)$ b) $O(n)$ c) $O(n \log n)$ d) $O(n^2)$ 23. Ω notation represents: a) worst case b) best/lower bound c) average d) tight bound 24. `for(i=1; i<=n; i++) for(j=1; j<=n; j+=i)` — dominant behavior is closest to: a) $O(n)$ b) $O(n \log n)$ c) $O(n^2)$ d) $O(\log n)$ 25. Space complexity of an in-place sort: a) $O(1)$ b) $O(n)$ c) $O(\log n)$ d) $O(n^2)$

Difficult (26–30) 26. $T(n) = T(n/2) + O(1)$ solves to: a) $O(1)$ b) $O(\log n)$ c) $O(n)$ d) $O(n \log n)$ 27. $T(n) = 2T(n/2) + O(n)$ solves to: a) $O(n)$ b) $O(n \log n)$ c) $O(n^2)$ d) $O(\log n)$ 28. Nested loop where inner halves each time: `for(i=0; i<n; i++) for(j=n; j>1; j/=2)`: a) $O(n)$ b) $O(n \log n)$ c) $O(n^2)$ d) $O(\log n)$ 29. Generating all subsets of n elements: a) $O(n)$ b) $O(n^2)$ c) $O(2^n)$ d) $O(n!)$ 30. Generating all permutations of n elements: a) $O(2^n)$ b) $O(n!)$ c) $O(n^2)$ d) $O(n \log n)$

✓ Answer Key — Topic 4

1-b · 2-c · 3-b · 4-c · 5-b · 6-a · 7-b · 8-c · 9-c · 10-b · 11-b · 12-b · 13-b · 14-b · 15-a · 16-c · 17-b · 18-b · 19-c · 20-c · 21-d · 22-b · 23-b · 24-b · 25-a · 26-b · 27-b · 28-b · 29-c · 30-b

Key explanations: **16** Triangular sum $n(n+1)/2 \rightarrow O(n^2)$. **19** Keep the dominant term $\rightarrow O(n^2)$. **24** Inner does n/i iterations; sum $n/1 + n/2 + \dots + n/n = n \cdot H_n \approx O(n \log n)$. **26** Halving with $O(1)$ work $\rightarrow O(\log n)$. **27** Master theorem $\rightarrow O(n \log n)$. **28** Outer $O(n) \times$ inner $O(\log n) = O(n \log n)$. **29** 2^n subsets. **30** $n!$ permutations.

Topic 5 — SQL

 **Why it matters:** SQL is the **#2 highest-yield topic** after OOP. **Every** target company tests it — Therap, Selise, WellDev, Brain Station 23, DataSoft especially. Expect 6–12 questions: joins, GROUP BY, aggregate functions, keys, and “what does this query output.” Master joins and you bank easy marks.

The clause execution order (critical)

You *write* SQL in one order but the DB *executes* it in another:

```
Written:  SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY
Executed: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY
```

💡 This is why you **can't use a column alias in WHERE** (SELECT runs after WHERE) but **can in ORDER BY**.

JOINS — the most-tested SQL concept

```
Table A      Table B
(INNER)      = rows matching in BOTH
(LEFT)       = ALL of A + matching B (NULLs where no match)
(RIGHT)      = ALL of B + matching A
(FULL)       = all of both, NULLs where no match
(CROSS)      = every combination (Cartesian product, A rows × B rows)
```

```
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.id;
```

Join	Returns
INNER	only matching rows in both
LEFT (OUTER)	all left rows + matched right (NULL if none)
RIGHT (OUTER)	all right rows + matched left
FULL (OUTER)	all rows from both
CROSS	Cartesian product (m × n rows)
SELF	table joined to itself (e.g., employee → manager)

💡 **INNER = intersection. LEFT keeps the left. To find “A without match in B”:** `LEFT JOIN ... WHERE b.id IS NULL`.

Aggregate functions + GROUP BY

`COUNT()`, `SUM()`, `AVG()`, `MIN()`, `MAX()`. Used with **GROUP BY** to summarize per group.

```
SELECT dept_id, COUNT(*) AS cnt, AVG(salary) AS avg_sal
FROM employees
GROUP BY dept_id
HAVING COUNT(*) > 5;      -- HAVING filters GROUPS; WHERE filters ROWS
```

🧠 **WHERE filters rows *before* grouping; HAVING filters groups *after*.** You can't use aggregate functions in WHERE — use HAVING.

COUNT(*) counts all rows; COUNT(col) ignores NULLs. Common trap.

Keys & constraints

Key	Meaning
Primary Key	uniquely identifies a row; unique + NOT NULL ; one per table
Foreign Key	references a PK in another table (enforces referential integrity)
Unique	no duplicates, but allows one NULL
Candidate Key	any column(s) that could be a PK
Composite Key	PK made of 2+ columns
Super Key	any set of columns that uniquely identifies a row

🧠 **Primary key = unique + NOT NULL. Unique = unique + allows NULL.**

Key SQL operators & clauses

- DISTINCT** — remove duplicate rows.
- IN (a,b,c)** / **BETWEEN x AND y** / **LIKE 'A%'** (% = any chars, _ = one char).
- NULL** handling: use **IS NULL** / **IS NOT NULL** (never **= NULL**). **Any arithmetic/comparison with NULL → NULL.**
- UNION** (removes duplicates) vs **UNION ALL** (keeps duplicates, faster).
- Subquery / nested query; correlated subquery (references outer query).
- ORDER BY col ASC|DESC**, **LIMIT n** (MySQL) / **TOP n** (SQL Server) / **ROWNUM** (Oracle).

DDL vs DML vs DCL vs TCL (asked directly)

Category	Commands
DDL (Data Definition)	CREATE, ALTER, DROP, TRUNCATE
DML (Data Manipulation)	SELECT, INSERT, UPDATE, DELETE
DCL (Data Control)	GRANT, REVOKE
TCL (Transaction Control)	COMMIT, ROLLBACK, SAVEPOINT

🧠 **DELETE vs TRUNCATE vs DROP:** DELETE = remove rows (DML, can WHERE, can rollback, keeps structure). TRUNCATE = remove **all** rows fast (DDL, no WHERE, resets, can't rollback in most DBs). DROP = delete the **whole table** (structure + data). **This is a top-5 SQL MCQ.**

Common mistakes & tricks

- ✗ `WHERE salary = NULL` → always false; use `IS NULL`.
- ✗ Using aggregate in WHERE → use **HAVING**.
- ✗ Alias in WHERE → not allowed (SELECT runs after WHERE).
- ✗ "COUNT(col) counts NULLs" → it **skips** NULLs; `COUNT(*)` counts all.
- ✗ "TRUNCATE can use WHERE" → **No**, it removes everything.
- 💡 **INNER=intersection, LEFT=all-left, HAVING=filter groups.**

Cheat sheet

```
Exec order: FROM→WHERE→GROUP BY→HAVING→SELECT→ORDER BY
JOINS: INNER=match both | LEFT=all left | RIGHT=all right | FULL=all | CROSS=m×n
WHERE=filter rows (pre-group) | HAVING=filter groups (post-group, aggregates)
PK = unique + NOT NULL | UNIQUE = unique + allows NULL | FK → references PK
COUNT(*)=all rows | COUNT(col)=non-NULL | NULL: use IS NULL
DELETE(DML,WHERE,rollback) | TRUNCATE(DDL,all,fast) | DROP(table gone)
DDL:CREATE/ALTER/DROP/TRUNCATE | DML:SELECT/INSERT/UPDATE/DELETE
UNION(distinct) vs UNION ALL(dupes) | LIKE %=many _=one
```

MCQs — attempt, then check key

Beginner (1–15) 1. Which returns only matching rows in both tables? a) LEFT JOIN b) INNER JOIN c) FULL JOIN d) CROSS JOIN 2. WHERE filters: a) groups b) rows c) columns d) tables 3. HAVING filters: a) rows before grouping b) groups after aggregation c) columns d) indexes 4. A primary key is: a) unique + nullable b) unique + NOT NULL c) allows duplicates d) always numeric 5. Which is an aggregate function? a) LIKE b) COUNT c) DISTINCT d) WHERE 6. UNION vs UNION ALL: UNION ALL: a) removes duplicates b) keeps duplicates c) sorts d) joins 7. Which command removes ALL rows but keeps the table, fast, no WHERE? a) DELETE b) DROP c) TRUNCATE d) REMOVE 8. Which is DDL? a) SELECT b) UPDATE c) CREATE d) INSERT 9. To test for NULL you use: a) = NULL b) IS NULL c) == NULL d) NULL() 10. `LIKE 'A%'` matches strings that: a) contain A b) start with A c) end with A d) equal A 11. CROSS JOIN produces: a) matching rows b) Cartesian product c) left rows d) no rows 12. Which removes duplicate rows in a result? a) DISTINCT b) UNIQUE c) GROUP d) COUNT 13. Foreign key enforces: a) uniqueness b) referential integrity c) sorting d) indexing 14. `COUNT(*)` counts: a) non-null only b) all rows c) distinct only d) columns 15. Which is DML? a) DROP b) GRANT c) UPDATE d) COMMIT

Intermediate (16–25) 16. Which JOIN returns all left-table rows even without a match? a) INNER b) LEFT c) RIGHT d) CROSS 17. `COUNT(col)` where col has NULLs: a) counts NULLs too b) ignores NULLs c) errors d) counts columns 18. You cannot use a SELECT column alias in: a) ORDER BY b) WHERE c) both d) neither 19. To find employees with no department: a) INNER JOIN b) LEFT JOIN ... WHERE dept IS NULL c) CROSS JOIN d) RIGHT JOIN only 20. UNIQUE constraint allows: a) no NULLs b) one NULL (typically) c) duplicates d) only integers 21. Which filters aggregated results? a) WHERE b) HAVING c) GROUP BY d) ORDER BY 22. TRUNCATE is classified as: a) DML b) DDL c) DCL d) TCL 23. A correlated subquery: a) runs once b) references the outer query per row c) is faster always d) is illegal 24. `SELECT dept, COUNT(*) FROM emp GROUP BY dept HAVING COUNT(*)>3` returns depts with: a) ≤3 emps b) >3 emps c) all d) exactly 3 25. Result of `5 + NULL` in SQL: a) 5 b) NULL c) 0 d) error

Difficult (26–30) 26. Given emp(id,name,mgr_id), to list each employee with their manager's name you use:
a) CROSS JOIN b) SELF JOIN c) UNION d) subquery only 27. Which correctly gets the 2nd highest salary? a)
MAX(salary) b) SELECT MAX(salary) FROM emp WHERE salary < (SELECT MAX(salary) FROM emp) c)
LIMIT 2 d) COUNT(salary) 28. WHERE dept='HR' GROUP BY dept — WHERE executes: a) after GROUP BY
b) before GROUP BY c) after SELECT d) never 29. INNER JOIN of A(3 rows) and B(4 rows) with no matches
returns: a) 12 rows b) 7 rows c) 0 rows d) 3 rows 30. CROSS JOIN of A(3 rows) and B(4 rows) returns: a) 7 b)
12 c) 0 d) 4

✅ Answer Key — Topic 5

1-b · 2-b · 3-b · 4-b · 5-b · 6-b · 7-c · 8-c · 9-b · 10-b · 11-b · 12-a · 13-b · 14-b · 15-c · 16-b · 17-b · 18-b · 19-b
· 20-b · 21-b · 22-b · 23-b · 24-b · 25-b · 26-b · 27-b · 28-b · 29-c · 30-b

Key explanations: **7** TRUNCATE = fast, all rows, DDL. **17** COUNT(col) ignores NULLs. **18** Alias unusable in WHERE (SELECT runs after WHERE), but fine in ORDER BY. **25** Any arithmetic with NULL → NULL. **26** Manager is in the same table → SELF JOIN. **27** Classic 2nd-highest-salary pattern. **29** No matches → INNER returns 0. **30** CROSS = 3×4 = 12.

Topic 6 — Database Management Systems (DBMS)

 **Why it matters:** DBMS theory pairs with SQL. Therap, Enosis, BJIT, DataSoft test **normalization, ACID, keys, and indexing**. These are conceptual — high-scoring if you memorize the definitions cleanly.

Normalization (the #1 DBMS topic)

Purpose: organize tables to **reduce redundancy** and **avoid anomalies** (insert, update, delete anomalies).

Form	Rule (plain English)
1NF	Atomic values only — no repeating groups/arrays in a cell
2NF	1NF + no partial dependency (non-key fully depends on the <i>whole</i> composite key)
3NF	2NF + no transitive dependency (non-key depends on another non-key)
BCNF	Stronger 3NF — every determinant is a candidate key

🧠 Memory line: “**1NF = atomic, 2NF = no partial, 3NF = no transitive, BCNF = every determinant is a key.**” 🧠 The classic mnemonic: “*The key, the whole key, and nothing but the key, so help me Codd.*” (2NF=whole key, 3NF=nothing but the key).

Denormalization = deliberately adding redundancy back for **read performance** (fewer joins). Trade-off vs data integrity.

ACID properties (transaction guarantees)

A – Atomicity → all-or-nothing (transaction fully completes or fully rolls back)
C – Consistency → DB moves from one valid state to another (constraints hold)
I – Isolation → concurrent transactions don't interfere
D – Durability → committed data survives crashes (persisted)

🧠 Bank transfer: debit + credit must **both** happen (Atomicity) or neither.

Keys (recap + DBMS angle)

- **Super key** $\supseteq$ **Candidate key** $\supseteq$ **Primary key**. A candidate key is a *minimal* super key.
- **Primary key**: chosen candidate key; unique + NOT NULL.
- **Foreign key**: references a PK; enforces **referential integrity**.
- **Composite key**: 2+ columns together form the key.

Indexing

An **index** speeds up **reads/searches** (like a book's index) but **slows down writes** (INSERT/UPDATE must update the index too) and uses extra space.

- **Clustered index:** determines the **physical order** of rows; **one per table** (usually the PK).
- **Non-clustered index:** separate structure pointing to rows; **many per table**.
- Typically implemented with **B-trees / B+ trees**.

🌸 **Index = faster reads, slower writes.** Great for columns in WHERE/JOIN/ORDER BY.

Transactions & concurrency (know the terms)

- **Deadlock:** two transactions each wait for a lock the other holds. DB detects and aborts one.
- **Isolation levels** (low→high): Read Uncommitted → Read Committed → Repeatable Read → Serializable. Higher = safer but slower.
- Concurrency problems: **Dirty read** (reading uncommitted data), **Non-repeatable read**, **Phantom read**.

SQL vs NoSQL (frequently asked)

SQL (Relational)	NoSQL
Tables, fixed schema	Flexible schema (document/key-value/graph/column)
ACID, strong consistency	Often BASE, eventual consistency
Vertical scaling	Horizontal scaling
MySQL, PostgreSQL, Oracle	MongoDB, Redis, Cassandra
Complex joins, structured data	Big data, rapid change, unstructured

DBMS vs File System (definition question)

DBMS advantages: reduced redundancy, data integrity/consistency, concurrent access control, security, backup/recovery, query language. A plain file system lacks these.

Common mistakes & tricks

- ❌ "3NF removes partial dependency" → that's **2NF**; 3NF removes **transitive**.
- ❌ "You can have many clustered indexes" → **one** per table.
- ❌ "Indexes speed up everything" → they **slow writes**.
- ❌ "ACID's C = concurrency" → C = **Consistency**; I = Isolation is concurrency.
- 🌸 **2NF = whole key (partial), 3NF = nothing but the key (transitive).**

Normalization: 1NF atomic | 2NF no partial dep | 3NF no transitive dep | BCNF determinant=key
ACID: Atomicity(all/nothing) Consistency(valid) Isolation(no interfere) Durability(persist)
Keys: super $\supseteq$ candidate $\supseteq$ primary; FK+PK (referential integrity)
Index: faster reads, slower writes; clustered=1/table(physical order), non-clustered=many
Isolation levels: ReadUncommitted<ReadCommitted<RepeatableRead<Serializable
SQL=schema+ACID+vertical | NoSQL=flexible+BASE+horizontal
Denormalization = add redundancy for read speed

MCQs — attempt, then check key

Beginner (1–15) 1. Normalization primarily reduces: a) speed b) redundancy c) security d) indexes 2. 1NF requires values to be: a) unique b) atomic c) numeric d) indexed 3. ACID's "A" stands for: a) Availability b) Atomicity c) Access d) Aggregation 4. ACID's "D" stands for: a) Data b) Durability c) Distinct d) Delete 5. A primary key is: a) nullable b) unique + NOT NULL c) always foreign d) duplicated 6. An index mainly speeds up: a) inserts b) reads/searches c) deletes d) backups 7. How many clustered indexes per table? a) 0 b) 1 c) many d) unlimited 8. Foreign key enforces: a) atomicity b) referential integrity c) indexing d) normalization 9. Which stores data in flexible/document form? a) MySQL b) MongoDB c) Oracle d) PostgreSQL 10. Atomicity means a transaction is: a) fast b) all-or-nothing c) isolated d) durable 11. Which is a relational DBMS? a) Redis b) MongoDB c) PostgreSQL d) Cassandra 12. A composite key consists of: a) one column b) two or more columns c) a foreign key only d) an index 13. Denormalization improves: a) integrity b) read performance c) normalization d) security 14. Dirty read means reading: a) old data b) uncommitted data c) deleted data d) indexed data 15. Which removes transitive dependency? a) 1NF b) 2NF c) 3NF d) 0NF

Intermediate (16–25) 16. 2NF removes: a) transitive dependency b) partial dependency c) atomic values d) all keys 17. "The whole key" refers to which normal form? a) 1NF b) 2NF c) 3NF d) BCNF 18. Higher isolation level generally means: a) faster + less safe b) slower + safer c) no locks d) no effect 19. A super key is: a) always minimal b) any set of columns uniquely identifying a row c) always a single column d) a foreign key 20. Indexes on a write-heavy table: a) always help b) can hurt write performance c) reduce storage d) enforce keys 21. BCNF requires every determinant to be: a) a foreign key b) a candidate key c) nullable d) numeric 22. Which is NOT an ACID property? a) Atomicity b) Consistency c) Scalability d) Durability 23. NoSQL databases typically scale: a) vertically b) horizontally c) not at all d) only with SQL 24. A candidate key is: a) any super key b) a minimal super key c) always composite d) a foreign key 25. Which anomaly does normalization prevent? a) syntax error b) update/insert/delete anomalies c) deadlock d) index bloat

Difficult (26–30) 26. Repeatable Read prevents but Read Committed allows: a) dirty reads b) non-repeatable reads c) durability d) atomicity 27. B+ tree indexes are preferred because: a) $O(n)$ search b) sorted + $O(\log n)$ range queries c) hash-based d) no disk use 28. In a table $PK(A,B)$, if C depends only on A, that is a: a) transitive dependency b) partial dependency c) full dependency d) trivial dependency 29. Serializable isolation prevents: a) only dirty reads b) dirty + non-repeatable + phantom reads c) nothing d) durability 30. A clustered index defines the table's: a) logical schema b) physical row order c) foreign keys d) normalization

✅ Answer Key — Topic 6

1-b · 2-b · 3-b · 4-b · 5-b · 6-b · 7-b · 8-b · 9-b · 10-b · 11-c · 12-b · 13-b · 14-b · 15-c · 16-b · 17-b · 18-b · 19-b · 20-b · 21-b · 22-c · 23-b · 24-b · 25-b · 26-b · 27-b · 28-b · 29-b · 30-b

Key explanations: **15/16** 3NF = transitive, 2NF = partial. **17** "Whole key" = 2NF. **22** Scalability is not an ACID property. **28** C depends on part of the composite key (A) → partial dependency (2NF violation). **29** Serializable is the strictest — prevents all three read anomalies. **30** Clustered index = physical order of rows.

Topic 7 — Operating Systems

 **Why it matters:** OS is a **core CS-fundamentals** section in nearly every BD trainee paper (Cefalo, TigerIT, Enosis, BJIT). Expect 4–8 MCQs on process/thread, scheduling, deadlock, memory, and synchronization.

Process vs Thread (top OS question)

Process = a program in execution with its **own memory space**. **Thread** = a lightweight unit *inside* a process that **shares** the process's memory.

	Process	Thread
Memory	Own private	Shared within process
Creation	Heavy	Light
Communication	IPC (pipes, sockets)	Shared memory (easy)
Crash	Isolated	Can crash whole process

💡 **Process = private memory; Thread = shared memory.** Threads → faster but need synchronization (race conditions).

Process states

```
NEW → READY ⇄ RUNNING → TERMINATED
      ↑           ↓
      WAITING ← (I/O; returns to READY, not RUNNING)
```

CPU Scheduling algorithms

Algorithm	Idea	Note
FCFS	first come first served	simple; convoy effect
SJF	shortest job first	best avg wait; starvation risk
Round Robin	fixed time quantum, rotate	fair; interactive; preemptive
Priority	highest priority first	starvation (fix: aging)

💡 **Round Robin = preemptive + fair (time slices).** **SJF = optimal avg waiting time.** Preemptive = OS can interrupt (RR, SRTF); non-preemptive = runs to completion (FCFS, basic SJF).

Deadlock (guaranteed question)

Four conditions — **all must hold** (Coffman): 1. **Mutual exclusion** 2. **Hold and wait** 3. **No preemption** 4. **Circular wait**

Break any one to prevent. Handling: prevention, avoidance (**Banker's algorithm**), detection & recovery.

🔴 **Deadlock (stuck forever in a cycle) ≠ Starvation (keeps getting skipped).**

Synchronization

- **Race condition:** outcome depends on thread timing on shared data.
- **Critical section:** code accessing shared resource — must be mutually exclusive.
- **Mutex:** lock, one thread at a time, owned. **Semaphore:** counter allowing up to N; binary semaphore $\approx$ mutex but not owned.

Memory management

- **Virtual memory:** each process gets a private address space; disk used as overflow (**swap**).
- **Paging:** fixed-size **pages** $\leftrightarrow$ **frames**; a **page fault** = accessing a page not in RAM. Constant faulting = **thrashing**.
- **Segmentation:** variable-size logical segments (code/stack/heap).
- **Fragmentation:** **Internal** (wasted space inside allocated block — paging) vs **External** (gaps between blocks — segmentation).

Other high-frequency facts

- **Kernel** = privileged core; **user space** = restricted apps; cross via **system calls**.
- **Zombie** = child finished, not reaped by parent. **Orphan** = parent died, child adopted by init (PID 1).
- **Context switch** = save one process's state, load another's — pure overhead.
- **Belady's anomaly:** more frames can cause more page faults (FIFO page replacement).
- Page replacement: **FIFO, LRU, Optimal**. LRU = replace least recently used.

Common mistakes & tricks

- ❌ "Threads have separate memory" → they **share** it.
- ❌ "Round Robin is non-preemptive" → it's **preemptive**.
- ❌ "Paging causes external fragmentation" → **internal**; segmentation → external.
- ❌ "Deadlock needs any one condition" → needs **all four**.
- 🟡 **SJF = best average waiting time. RR = fairness.**

Cheat sheet

```
Process=own memory | Thread=shared memory (needs sync)
States: New→Ready→Running→(Waiting)→Terminated
Scheduling: FCFS(convoy) | SJF(best avg wait, starvation) | RR(preemptive,fair) | Priority(aging)
Deadlock 4 conds: mutual exclusion, hold&wait, no preemption, circular wait (ALL)
Mutex=1 owned | Semaphore=N counter | Race condition on shared data
Virtual memory+paging(pages↔frames), page fault, thrashing
Fragmentation: internal(paging) vs external(segmentation)
Zombie=child dead unreaped | Orphan=parent dead (init adopts)
Page replacement: FIFO/LRU/Optimal; Belady's anomaly (FIFO)
```

MCQs — attempt, then check key

Beginner (1–15) 1. A thread differs from a process in that threads: a) have own memory b) share process memory c) can't run concurrently d) are slower to create 2. Which scheduling is preemptive and time-sliced? a) FCFS b) SJF (basic) c) Round Robin d) Priority (non-preemptive) 3. How many deadlock conditions must hold? a) 1 b) 2 c) 3 d) 4 4. A zombie process is one that: a) is running b) finished but not reaped c) has no parent d) is sleeping 5. An orphan process is adopted by: a) the kernel b) init/PID 1 c) its sibling d) no one 6. Which gives the best average waiting time? a) FCFS b) SJF c) RR d) Priority 7. A page fault occurs when: a) disk is full b) page not in RAM c) CPU fails d) process ends 8. Mutex allows how many threads in critical section? a) 0 b) 1 c) N d) unlimited 9. Context switch is: a) useful computation b) pure overhead c) a system call type d) a deadlock 10. Which uses fixed-size blocks? a) segmentation b) paging c) swapping d) fragmentation 11. Virtual memory uses disk as: a) cache b) overflow (swap) c) registers d) kernel 12. Process with own memory space; thread with: a) own memory b) shared memory c) no memory d) kernel memory 13. FCFS suffers from: a) starvation b) convoy effect c) thrashing d) deadlock only 14. Kernel runs in: a) user space b) privileged/kernel space c) swap d) cache 15. System calls let user programs: a) access hardware directly b) request kernel services c) skip the OS d) run as root always

Intermediate (16–25) 16. Paging causes which fragmentation? a) external b) internal c) both equally d) none 17. Segmentation causes which fragmentation? a) internal b) external c) none d) both 18. Constant page faulting is called: a) swapping b) thrashing c) paging d) starvation 19. Priority scheduling's starvation is fixed by: a) aging b) paging c) locking d) swapping 20. A semaphore differs from a mutex because it: a) allows N and isn't owned b) is always binary c) is faster d) can't block 21. Deadlock vs starvation: deadlock is: a) skipping a process b) circular wait, stuck forever c) fast d) preemptive 22. LRU page replacement evicts the: a) newest page b) least recently used page c) largest page d) first loaded 23. Belady's anomaly affects: a) LRU b) Optimal c) FIFO d) all equally 24. Which is NOT a deadlock condition? a) mutual exclusion b) preemption allowed c) hold and wait d) circular wait 25. Which memory issue is "internal fragmentation"? a) gaps between blocks b) wasted space inside an allocated block c) no free memory d) page fault

Difficult (26–30) 26. Banker's algorithm is used for deadlock: a) detection b) avoidance c) recovery d) prevention only 27. In RR, if the time quantum is very large, it behaves like: a) SJF b) FCFS c) Priority d) LRU 28. Two threads incrementing a shared counter without a lock cause a: a) deadlock b) race condition c) page fault d) context switch 29. Which scheduling can cause starvation of long jobs? a) FCFS b) Round Robin c) SJF/SRTF d) none 30. A process moving from Running to Waiting typically means it: a) finished b) needs I/O c) was created d) got the CPU

✓ Answer Key — Topic 7

1-b · 2-c · 3-d · 4-b · 5-b · 6-b · 7-b · 8-b · 9-b · 10-b · 11-b · 12-b · 13-b · 14-b · 15-b · 16-b · 17-b · 18-b · 19-a · 20-a · 21-b · 22-b · 23-c · 24-b · 25-b · 26-b · 27-b · 28-b · 29-c · 30-b

Key explanations: **16/17** Paging → internal fragmentation; segmentation → external. **23** Belady's anomaly is a FIFO phenomenon. **26** Banker's algorithm = deadlock *avoidance*. **27** RR with a huge quantum → each process finishes in one slice → behaves like FCFS. **29** SJF/SRTF starves long jobs when short jobs keep arriving.

Topic 8 — Computer Networks

 **Why it matters:** Networking basics appear in most BD trainee papers (Cefalo, TigerIT, WellDev, Selise). Focus on **OSI/TCP-IP, TCP vs UDP, HTTP status codes, DNS, and ports** — the same handful of facts repeat.

OSI Model (7 layers)

```
7 Application    - HTTP, DNS, FTP, SMTP
6 Presentation  - encryption, compression (TLS, JPEG)
5 Session       - connections/sessions
4 Transport     - TCP, UDP, ports
3 Network       - IP, routers
2 Data Link     - MAC, switches, Ethernet
1 Physical      - cables, signals
```

💡 Mnemonic: **“All People Seem To Need Data Processing”** (top→bottom). 💡 The tested layers: **L7 HTTP, L4 TCP/UDP, L3 IP, L2 MAC/switch**.

TCP/IP model = 4 layers: Application, Transport, Internet, Network Access.

TCP vs UDP (top networking MCQ)

	TCP	UDP
Connection	Yes (3-way handshake)	No
Reliable/ordered	Yes	No (best-effort)
Speed	Slower	Faster
Use	web, email, file transfer	video, gaming, DNS, VoIP

3-way handshake: SYN → SYN-ACK → ACK. 💡 **TCP = reliable/slow; UDP = fast/unreliable.**

IP addressing

- **IPv4** = 32-bit (e.g., 192.168.1.1), ~4.3 billion addresses. **IPv6** = 128-bit.
- **Private ranges:** 10.x, 172.16–31.x, 192.168.x. **Loopback:** 127.0.0.1.
- **Classes:** A (large), B, C (small). Subnet mask separates network/host.
- **NAT:** many private IPs share one public IP.

Key ports (memorize)

Port	Service		Port	Service
20/21	FTP		443	HTTPS
22	SSH		25	SMTP
23	Telnet		53	DNS
80	HTTP		3306	MySQL

DNS, DHCP, HTTP

- **DNS**: name → IP (port 53). Returns an **IP address**, not the page.
- **DHCP**: auto-assigns IP (DORA: Discover, Offer, Request, Ack).
- **HTTP** is **stateless**; port 80. **HTTPS** = HTTP + TLS, port 443 (encryption + certificate identity).

HTTP status codes (guaranteed MCQ)

1xx Info | 2xx Success | 3xx Redirect | 4xx Client error | 5xx Server error
200 OK 201 Created 301 Moved 302 Found 304 Not Modified
400 Bad Request 401 Unauthorized 403 Forbidden 404 Not Found 429 Too Many
500 Internal Error 502 Bad Gateway 503 Unavailable 504 Gateway Timeout

🔴 4xx = client's fault, 5xx = server's fault. 401 = not authenticated; 403 = authenticated but forbidden.

Devices & concepts

Device	Layer	Role
Hub	1	dumb broadcast to all ports
Switch	2	forwards by MAC (smart)
Router	3	routes between networks by IP
Firewall	3/4	filters traffic by rules

- **Bandwidth** = capacity; **Latency** = delay. **Packet** = unit of data. **Protocol** = rules.
- **HTTP vs HTTPS**: HTTPS encrypts via TLS + verifies identity with a certificate.

Common mistakes & tricks

- ❌ "UDP is reliable" → **best-effort**, no guarantee.
- ❌ "DNS returns the web page" → returns an **IP**.
- ❌ "Switch works at Layer 3" → **Layer 2** (router = L3).
- ❌ "401 = forbidden" → 401 = **unauthenticated**; 403 = forbidden.
- 🟡 **HTTP=80, HTTPS=443, SSH=22, DNS=53, FTP=21.**

```
OSI 7: App-Pres-Sess-Trans-Net-DataLink-Phys ("All People Seem To Need Data Processing")
TCP=reliable/ordered/slow (handshake) | UDP=fast/best-effort
Ports: 21 FTP, 22 SSH, 25 SMTP, 53 DNS, 80 HTTP, 443 HTTPS, 3306 MySQL
DNS→IP (port 53) | DHCP=DORA | HTTP stateless(80) | HTTPS=TLS(443)
Status: 2xx ok, 3xx redirect, 4xx client, 5xx server | 401 unauth, 403 forbidden, 404 notfound
Hub(L1) < Switch(L2, MAC) < Router(L3, IP) | NAT=private→public
IPv4=32bit, IPv6=128bit, loopback 127.0.0.1
```

MCQs — attempt, then check key

Beginner (1–15) 1. How many layers in the OSI model? a) 4 b) 5 c) 7 d) 8 2. TCP is: a) connectionless b) connection-oriented/reliable c) fast/unreliable d) a device 3. UDP is used for: a) email b) file transfer c) live video/gaming d) banking 4. HTTP default port: a) 21 b) 80 c) 443 d) 22 5. HTTPS default port: a) 80 b) 443 c) 22 d) 8080 6. DNS resolves: a) IP to MAC b) name to IP c) port to service d) page to server 7. 404 status code means: a) success b) redirect c) not found d) server error 8. Which works at Layer 2 (MAC)? a) hub b) switch c) router d) firewall 9. A router works at OSI layer: a) 1 b) 2 c) 3 d) 7 10. SSH uses port: a) 21 b) 22 c) 23 d) 25 11. TCP 3-way handshake is: a) SYN, ACK, FIN b) SYN, SYN-ACK, ACK c) GET, POST, PUT d) DORA 12. IPv4 address size: a) 16-bit b) 32-bit c) 64-bit d) 128-bit 13. 5xx status codes indicate: a) client error b) server error c) redirect d) success 14. Which is stateless? a) TCP b) HTTP c) FTP session d) SSH 15. DHCP is used to: a) resolve names b) auto-assign IP addresses c) encrypt traffic d) route packets

Intermediate (16–25) 16. 401 vs 403: 401 means: a) forbidden b) not authenticated c) not found d) server error 17. IPv6 address size: a) 32-bit b) 64-bit c) 128-bit d) 256-bit 18. Loopback address: a) 0.0.0.0 b) 127.0.0.1 c) 192.168.1.1 d) 255.255.255.255 19. Which protocol guarantees ordered delivery? a) UDP b) TCP c) IP d) ICMP 20. NAT allows: a) name resolution b) many private IPs to share a public IP c) encryption d) routing tables 21. Port 53 is: a) HTTP b) DNS c) SMTP d) SSH 22. A switch forwards frames using: a) IP address b) MAC address c) port number d) hostname 23. HTTPS adds which over HTTP? a) speed b) TLS encryption + identity c) statelessness d) new ports only 24. 301 status code: a) temporary redirect b) permanent redirect c) not modified d) created 25. Which layer does IP belong to? a) 2 b) 3 c) 4 d) 7

Difficult (26–30) 26. DNS primarily uses which transport protocol? a) TCP only b) UDP (port 53) c) HTTP d) ICMP 27. Bandwidth vs latency: latency is: a) capacity b) delay c) packet size d) protocol 28. A 502 Bad Gateway typically means: a) client sent bad data b) upstream/backend returned invalid response c) not found d) unauthorized 29. Which is a private IP range? a) 8.8.8.8 b) 192.168.0.0/16 c) 172.32.0.1 d) 11.0.0.1 30. Subnet mask 255.255.255.0 gives how many host addresses (usable)? a) 254 b) 256 c) 1024 d) 512

✓ Answer Key — Topic 8

1-c · 2-b · 3-c · 4-b · 5-b · 6-b · 7-c · 8-b · 9-c · 10-b · 11-b · 12-b · 13-b · 14-b · 15-b · 16-b · 17-c · 18-b · 19-b · 20-b · 21-b · 22-b · 23-b · 24-b · 25-b · 26-b · 27-b · 28-b · 29-b · 30-a

Key explanations: **16** 401 = unauthenticated (log in); 403 = authenticated but forbidden. **26** DNS mostly uses UDP port 53 (TCP for large/zone transfers). **28** 502 = a proxy/gateway got an invalid response from the backend. **30** /24 → 256 addresses – 2 (network + broadcast) = 254 usable hosts.

Topic 9 — REST APIs

 **Why it matters:** WellDev, Selise, and web-focused teams test REST basics — HTTP methods, status codes, statelessness. Usually 2–4 MCQs. Easy marks if you know the verbs.

REST essentials

REST = an architecture style for web APIs built around **resources** (URLs) manipulated with **HTTP methods**, exchanging usually **JSON**. Key trait: **stateless** — each request carries all needed info; the server stores no client session between requests.

HTTP methods (know idempotency & safety)

Method	Purpose	Idempotent?	Safe (read-only)?
GET	read	✓	✓
POST	create	✗	✗
PUT	replace/update (full)	✓	✗
PATCH	partial update	⚠ not always	✗
DELETE	remove	✓	✗

 **Idempotent** = doing it multiple times = same result as once (GET, PUT, DELETE). **POST is NOT idempotent** (twice = two resources → double submit). This is the #1 REST trap.

REST design rules

- Resource URLs use **nouns**, not verbs: `/users/42` ✓, not `/getUser?id=42` ✗.
- Use HTTP methods for actions, status codes for results.
- Stateless, cacheable, client-server separation, uniform interface.

GET	<code>/users</code>	→ list	POST	<code>/users</code>	→ create
GET	<code>/users/42</code>	→ read one	PUT	<code>/users/42</code>	→ replace
PATCH	<code>/users/42</code>	→ partial	DELETE	<code>/users/42</code>	→ delete

Status codes (same as Networks §): 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Server Error.

REST vs SOAP (occasionally asked)

REST = lightweight, JSON, over HTTP, flexible. SOAP = XML, strict contract (WSDL), heavier. REST dominates modern web APIs.

Topic 10 — Git & Version Control

 **Why it matters:** Practical dev knowledge. 2–4 MCQs on basic commands, merge vs rebase, and the staging area. Every company assumes Git literacy.

The 3 areas + core commands

```
Working Dir --add--> Staging Area --commit--> Local Repo --push--> Remote
<-----pull-----<
```

Command	Does
<code>git clone</code>	copy a remote repo
<code>git status</code>	show changes
<code>git add</code>	stage changes
<code>git commit -m</code>	save staged changes locally
<code>git push</code>	upload to remote
<code>git pull</code>	fetch + merge from remote
<code>git fetch</code>	download only (no merge)
<code>git branch / git checkout -b</code>	list / create+switch branch
<code>git merge</code>	combine branches
<code>git stash</code>	shelve uncommitted changes

Key concepts

- **fetch vs pull:** fetch = download only; **pull = fetch + merge**.
- **merge vs rebase:** merge keeps history + a merge commit (branchy); **rebase** rewrites history to be linear. 🚫 **Never rebase shared/pushed history.**
- **reset vs revert:** `revert` = new commit that undoes (safe on shared); `reset` = move pointer back (rewrites history, local only). `reset --hard` discards changes.
- **Merge conflict:** two branches edit the same lines. Markers `<<<<<<< ===== >>>>>>>`; edit to correct version, `git add`, `git commit`.
- **HEAD** = pointer to current commit/branch.

Common mistakes & tricks

- ❌ "pull = fetch only" → **fetch + merge**.
- ❌ "commit uploads to GitHub" → commit is **local**; `push` uploads.
- ❌ Rebasing a shared branch → breaks teammates.
- 🚫 **Order:** `add` → `commit` → `push`. **Pull before you start.**

Combined cheat sheet (REST + Git)

```
REST: resources+HTTP methods, STATELESS, JSON
GET(read,idempotent) POST(create,NOT idempotent) PUT(replace,idempotent) DELETE(idempotent)
Idempotent = repeat = same result | URLs = nouns not verbs
GIT: add→commit(local)→push | pull=fetch+merge | fetch=download only
merge=history+mergecommit | rebase=linear(rewrites, never on shared)
revert=safe undo | reset=rewrite(local) | stash=shelve | HEAD=current commit
Conflict: <<<< mine ==== theirs >>>> → edit, add, commit
```

MCQs — attempt, then check key

Beginner (1–15) 1. REST APIs are typically: a) stateful b) stateless c) offline d) binary-only 2. Which HTTP method reads data? a) POST b) GET c) DELETE d) PUT 3. Which method creates a resource? a) GET b) POST c) PATCH d) HEAD 4. Which is idempotent? a) POST b) GET c) neither d) only PATCH 5. REST usually exchanges data in: a) XML only b) JSON c) CSV d) binary 6. Which command stages changes? a) git commit b) git add c) git push d) git pull 7. `git pull` equals: a) fetch only b) fetch + merge c) push + merge d) clone 8. Which uploads commits to the remote? a) git add b) git commit c) git push d) git status 9. A good REST URL for a user is: a) /getUser?id=1 b) /users/1 c) /user/get/1 d) /fetchUser 10. 201 status code means: a) OK b) Created c) Not Found d) Server Error 11. Which shelves uncommitted changes? a) git stash b) git drop c) git hide d) git save 12. DELETE method is: a) not idempotent b) idempotent c) safe/read-only d) a Git command 13. Which downloads without merging? a) git pull b) git fetch c) git push d) git commit 14. HTTP methods represent: a) resources b) actions on resources c) status codes d) URLs 15. `git commit` saves changes: a) to the remote b) to local repo c) to staging only d) nowhere

Intermediate (16–25) 16. POST is not idempotent because sending it twice: a) errors b) creates two resources c) does nothing d) is cached 17. merge vs rebase: rebase produces: a) branchy history b) linear history c) no history d) a merge commit 18. Which should you NOT rebase? a) local branch b) shared/pushed history c) a new branch d) an old commit 19. Which safely undoes a commit on a shared branch? a) reset – hard b) revert c) rebase d) checkout 20. Merge conflict markers include: a) /* */ b) <<<<<< ===== >>>>>> c) ### d) – ++ 21. PUT vs POST: PUT is: a) create, not idempotent b) replace, idempotent c) read-only d) partial update 22. 401 vs 403 in a REST API: 401 means: a) forbidden b) unauthenticated c) not found d) created 23. Statelessness means the server: a) stores session per client b) keeps no client state between requests c) never responds d) caches everything 24. `git fetch` then `git merge` equals: a) git push b) git pull c) git clone d) git stash 25. Which is a Git working area? a) staging area b) heap c) stack d) kernel

Difficult (26–30) 26. A double-clicked “Submit” that charges twice is caused by which method’s nature? a) GET b) POST (non-idempotent) c) PUT d) DELETE 27. `git reset --hard HEAD~1`: a) undoes commit, keeps changes b) undoes commit AND discards changes c) creates a commit d) pushes 28. PATCH differs from PUT by: a) being idempotent always b) doing a partial update c) reading data d) deleting 29. Which REST constraint enables caching and scalability? a) statefulness b) statelessness c) SOAP d) sessions 30. HEAD in Git refers to: a) the first commit b) the current commit/branch pointer c) the remote d) the staging area

Answer Key — Topics 9 & 10

1-b · 2-b · 3-b · 4-b · 5-b · 6-b · 7-b · 8-c · 9-b · 10-b · 11-a · 12-b · 13-b · 14-b · 15-b · 16-b · 17-b · 18-b · 19-b · 20-b · 21-b · 22-b · 23-b · 24-b · 25-a · 26-b · 27-b · 28-b · 29-b · 30-b

Key explanations: **4/16** GET/PUT/DELETE idempotent; POST not (repeat = duplicate). **19** revert is safe on shared branches; reset rewrites history. **26** Non-idempotent POST → double submit double-charges. **27** `--hard` discards working changes (destructive). **29** Statelessness lets any server handle any request → scalable + cacheable.

Topic 11 — Design Patterns (common ones only)

 **Why it matters:** Only a few patterns appear in trainee MCQs — usually **Singleton**, **Factory**, **Observer**, plus the **SOLID** principles. 1–3 questions. Don't over-study; know the intent of each.

The three categories

Creational	– how objects are created	(Singleton, Factory, Builder, Prototype)
Structural	– how objects are composed	(Adapter, Decorator, Facade, Proxy)
Behavioral	– how objects interact	(Observer, Strategy, Iterator, Command)

The patterns you must know

Singleton (Creational) — ensures a class has **only one instance** with a global access point. Uses: config, logging, DB connection pool.

```
class Singleton {
    private static Singleton instance;
    private Singleton() {} // private ctor blocks 'new'
    public static Singleton getInstance() {
        if (instance == null) instance = new Singleton();
        return instance;
    }
}
```

 **Private constructor + static getInstance() = Singleton.** (Thread-safety needs synchronization / eager init.)

Factory (Creational) — a method/class that **creates objects without exposing the exact class** to the caller. `ShapeFactory.create("circle")` returns a `Shape`. Decouples creation from use.

Observer (Behavioral) — a **one-to-many** dependency: when the subject changes, all observers are notified automatically. Uses: event systems, pub/sub, UI listeners, notifications.

Strategy (Behavioral) — define a family of interchangeable algorithms, select one at runtime (e.g., different sort/payment strategies).

Decorator (Structural) — add behavior to an object dynamically by wrapping it (e.g., `BufferedReader(new FileReader( ... ))`).

Adapter (Structural) — makes incompatible interfaces work together (a “translator”).

MVC (architectural) — Model (data), View (UI), Controller (logic). Separates concerns.

SOLID principles (frequently asked as a set)

Letter	Principle	One line
S	Single Responsibility	one class, one reason to change
O	Open/Closed	open for extension, closed for modification
L	Liskov Substitution	subtypes replaceable for base types
I	Interface Segregation	many small interfaces > one fat interface
D	Dependency Inversion	depend on abstractions, not concretions

Common mistakes & tricks

- ✗ "Singleton allows many instances" → **exactly one**.
- ✗ "Factory exposes the concrete class" → it **hides** it.
- ✗ Confusing Observer (notify many) with Strategy (swap algorithm).
- 🧠 **Singleton = one; Factory = create-without-new; Observer = notify subscribers.**
- 🧠 **SOLID = S.O.L.I.D** (5 principles) — know the acronym.

Cheat sheet

```
Categories: Creational | Structural | Behavioral
Singleton = one instance (private ctor + static getInstance)
Factory = create objects without exposing concrete class
Observer = one-to-many notify (pub/sub, events)
Strategy = interchangeable algorithms at runtime
Decorator = wrap to add behavior | Adapter = interface translator
MVC = Model-View-Controller
SOLID: Single-resp, Open/closed, Liskov, Interface-seg, Dependency-inversion
```

MCQs — attempt, then check key

Beginner (1–12) 1. Singleton ensures: a) many instances b) exactly one instance c) no instances d) two instances 2. Which enforces a single instance? a) public constructor b) private constructor + static getInstance c) many constructors d) interface 3. Factory pattern is a: a) structural pattern b) creational pattern c) behavioral pattern d) architectural pattern 4. Observer pattern is: a) one-to-one b) one-to-many notification c) creational d) a database pattern 5. Which category does Singleton belong to? a) creational b) structural c) behavioral d) functional 6. SOLID's "S" is: a) Singleton b) Single Responsibility c) Strategy d) Structural 7. Which adds behavior by wrapping? a) Adapter b) Decorator c) Singleton d) Factory 8. Adapter pattern is: a) creational b) structural (interface translator) c) behavioral d) singleton 9. MVC stands for: a) Model-View-Controller b) Multi-View-Class c) Model-Value-Cache d) Main-View-Core 10. Factory hides: a) the interface b) the concrete class being created c) the method name d) nothing 11. Which is a behavioral pattern? a) Singleton b) Factory c) Observer d) Adapter 12. Pub/sub event systems typically use: a) Singleton b) Observer c) Factory d) Adapter

Intermediate (13–20) 13. Open/Closed principle: classes should be open for ___ and closed for ___: a) reading/writing b) extension/modification c) input/output d) creation/deletion 14. Strategy pattern lets you: a) create one instance b) swap algorithms at runtime c) wrap objects d) translate interfaces 15. Dependency Inversion says depend on: a) concrete classes b) abstractions c) singletons d) globals 16. Which pattern would `BufferedReader(new FileReader(f))` illustrate? a) Singleton b) Decorator c) Observer d) Factory 17. Liskov Substitution: subtypes should be: a) faster b) substitutable for their base type c) final d) abstract 18. A logging/config class typically uses which pattern? a) Observer b) Singleton c) Strategy d) Adapter 19. Which is NOT a creational pattern? a) Singleton b) Factory c) Builder d) Observer 20. Interface Segregation prefers: a) one large interface b) many small focused interfaces c) no interfaces d) abstract classes only

✓ Answer Key — Topic 11

1-b · 2-b · 3-b · 4-b · 5-a · 6-b · 7-b · 8-b · 9-a · 10-b · 11-c · 12-b · 13-b · 14-b · 15-b · 16-b · 17-b · 18-b · 19-d · 20-b

Key explanations: **2** Private constructor prevents `new` ; static method returns the sole instance. **16** Wrapping streams to add buffering = Decorator. **19** Observer is behavioral, not creational.

Topic 12 — Basic System Design

 **Why it matters:** Trainee MCQs only test **basic** system-design vocabulary — scaling, load balancing, caching, database types. 1–3 conceptual questions. You won't design Twitter; you must recognize the terms.

Core vocabulary

Scalability — the key distinction:

Vertical scaling (scale UP) = bigger machine (more CPU/RAM). Simple, has a ceiling.
Horizontal scaling (scale OUT) = more machines. Harder, but near-limitless + fault-tolerant.

🟡 **Vertical = upgrade one server; Horizontal = add more servers.**

Load balancer — distributes traffic across multiple servers for **scalability + availability**; health-checks and removes dead servers. Algorithms: round robin, least connections.

Caching — store frequently-used data in fast memory (e.g., **Redis, Memcached**) to reduce DB load and latency. Trade-off: **stale data**. Cache invalidation is famously hard.

Database scaling: - **Replication**: copy data to multiple servers (read scaling, availability). Master-slave / primary-replica. - **Sharding/Partitioning**: split data across servers by key (write scaling). - **CDN** (Content Delivery Network): caches static content near users geographically.

Availability terms: - **Latency** (delay) vs **Throughput** (requests/second). - **High availability**: redundancy, no single point of failure. - **Fault tolerance**: system keeps working despite component failures.

CAP theorem (occasionally asked)

A distributed system can guarantee at most **2 of 3**: **C**onsistency, **A**vailability, **P**artition tolerance. Since network partitions happen, you effectively choose **CP** or **AP**. 🟡 **Consistency, Availability, Partition tolerance — pick 2.**

Architecture styles

- **Monolith** = one deployable unit (simple, but scales as a whole).
- **Microservices** = many small independent services (scalable, complex, network overhead).
- **Stateless services** scale horizontally easily (no per-server session).

Common patterns to recognize

- **Message queue** (Kafka, RabbitMQ) — decouples producers/consumers, handles spikes asynchronously.
- **Rate limiting** — cap requests per client (protect against abuse; returns 429).
- **API gateway / reverse proxy** — single entry point in front of services.

Common mistakes & tricks

- ❌ "Vertical scaling = more servers" → that's **horizontal**.
- ❌ "Caching guarantees fresh data" → risk of **stale** data.
- ❌ "CAP lets you have all three" → at most **two**.
- 🟡 **Scale out = horizontal = more boxes = fault tolerant.**
- 🟡 **Cache = speed; Load balancer = spread; Replication = read/availability; Sharding = write scaling.**

Cheat sheet

```
Vertical scaling=bigger machine | Horizontal scaling=more machines(fault tolerant)
Load balancer=spread traffic+health checks | Cache(Redis)=fast reads, risk stale
Replication=copies(read/availability) | Sharding=split by key(write scaling)
CDN=static content near users | Latency=delay, Throughput=rate
CAP: pick 2 of Consistency/Availability/Partition-tolerance
Monolith(1 unit) vs Microservices(many, complex) | Stateless scales easily
Message queue=async decouple | Rate limit→429 | API gateway=single entry
```

MCQs — attempt, then check key

Beginner (1–12) 1. Adding more servers is called: a) vertical scaling b) horizontal scaling c) caching d) sharding 2. Upgrading one server's CPU/RAM is: a) horizontal b) vertical scaling c) replication d) sharding 3. A load balancer's main job: a) store data b) distribute traffic across servers c) encrypt d) cache pages 4. Which is a caching system? a) MySQL b) Redis c) Kafka d) Nginx 5. Caching mainly reduces: a) security b) latency and DB load c) redundancy d) accuracy 6. CAP theorem: you can guarantee at most: a) 1 b) 2 of 3 c) all 3 d) 4 7. A CDN caches content: a) in the database b) near users geographically c) in the CPU d) in RAM only 8. Microservices are: a) one big unit b) many small independent services c) a database d) a cache 9. Replication copies data for: a) write scaling only b) read scaling + availability c) encryption d) sharding 10. Sharding splits data to improve: a) reads only b) write scaling c) caching d) security 11. Stateless services scale: a) poorly b) easily horizontally c) only vertically d) never 12. Which returns HTTP 429? a) load balancing b) rate limiting c) caching d) sharding

Intermediate (13–20) 13. The main downside of caching is: a) slower reads b) stale/inconsistent data c) more DB load d) higher latency 14. Message queues (Kafka/RabbitMQ) provide: a) synchronous calls b) async decoupling of producers/consumers c) caching d) load balancing only 15. Which improves availability by removing single points of failure? a) monolith b) redundancy c) caching d) sharding 16. Throughput measures: a) delay b) requests per second c) storage d) bandwidth cost 17. CP vs AP systems choose to sacrifice: A for CP means giving up: a) consistency b) availability c) partition tolerance d) performance 18. Horizontal scaling is preferred for large systems because it is: a) simpler b) near-limitless + fault-tolerant c) cheaper always d) stateful 19. An API gateway acts as: a) a database b) a single entry point/reverse proxy c) a cache only d) a message queue 20. Monolith vs microservices: microservices add: a) simplicity b) network/operational complexity c) one deploy unit d) no scaling

✅ Answer Key — Topic 12

1-b · 2-b · 3-b · 4-b · 5-b · 6-b · 7-b · 8-b · 9-b · 10-b · 11-b · 12-b · 13-b · 14-b · 15-b · 16-b · 17-b · 18-b · 19-b · 20-b

Key explanations: **6** CAP → at most 2 of 3. **13** Caching risks serving stale data (invalidation problem). **17** CP sacrifices Availability during a partition to stay consistent. **18** Horizontal scaling adds machines → fault tolerance + near-limitless growth.

Topic 13 — CSE Fundamentals & Code Output

 **Why it matters:** BD trainee papers pack in a grab-bag of “core CS” and **output-prediction** questions (C/C++/Java/Python/JS). These decide borderline scores. This section is pure high-frequency facts + the classic tricky snippets.

Number systems (guaranteed 2–4 questions)

```
Binary(2)  Decimal(10)  Hex(16)
Powers of 2: 1 2 4 8 16 32 64 128 256 512 1024
10102 = 8+0+2+0 = 1010      |      25510 = 111111112 = FF16
1 byte = 8 bits → values 0..255 (28) | 1 KB = 1024 bytes
```

Convert 45 to binary: $45 = 32 + 8 + 4 + 1 = 101101_2$. **Hex digit = 4 bits.** A=10, B=11, C=12, D=13, E=14, F=15.  **$2^{10}=1024$, $2^8=256$, $2^6=64$.** Memorize powers of 2 up to 1024.

Data & memory

- **Stack** = local variables, function calls, automatic, fast, LIFO (stack overflow). **Heap** = dynamic allocation (`new` / `malloc`), manual/GC, larger (memory leak).
- **Value type vs Reference type:** primitives (int, char) hold the value; objects/arrays hold a **reference** (pointer).
- **Pass by value** (copy) vs **pass by reference** (alias). Java is **always pass-by-value** — but for objects, the *reference* is passed by value (so you can mutate the object, not reassign it).
- **Static typing** (Java, C++ — types checked at compile time) vs **Dynamic typing** (Python, JS — at runtime).
- **Compiled** (C, C++ — to machine code) vs **Interpreted** (Python, JS) vs **Hybrid/bytecode** (Java → JVM, C# → CLR).

Bitwise & operators

```
& AND | OR ^ XOR ~ NOT << left shift(x2) >> right shift(÷2)
5 & 3 = 1 | 5 | 3 = 7 | 5 ^ 3 = 6 | 1 << 3 = 8 | x ^ x = 0
```

 **Left shift by n = multiply by 2^n . XOR of a number with itself = 0.** `n & (n-1)` clears the lowest set bit (used to check power of 2: `n & (n-1) == 0`).

Classic OUTPUT-prediction traps (study every one)

Integer division (C/C++/Java):

```
System.out.println(5/2);      // → 2   (integer division, not 2.5)
System.out.println(5.0/2);    // → 2.5 (one operand double)
System.out.println(5%2);      // → 1   (modulo)
System.out.println(7/2*2);    // → 6   (7/2=3, *2=6, left-to-right)
```

Post vs pre increment:

```
int i=5; System.out.println(i++); // → 5 (use then increment), i is now 6
int j=5; System.out.println(++j); // → 6 (increment then use)
int a=5; int b=a++ + ++a;        // b = 5 + 7 = 12, a=7
```

Java String immutability / == vs equals:

```
String a="hi", b="hi"; a==b        // → true (string pool, same reference)
String c=new String("hi"); a==c    // → false (different object)
a.equals(c)                        // → true (value comparison)
```

👉 Use `.equals()` for String content; `==` compares references (objects).

Java array default values: `int[] x = new int[3];` → `{0,0,0}`. Object arrays → `null`.

Python quirks:

```
print(5//2)    # → 2 (floor division)    print(5/2) # → 2.5 (true division)
print(2**3)    # → 8 (power)              print("ab"*3) # → ababab
print(bool(0), bool(""), bool([]))        # → False False False (falsy)
a=[1,2,3]; b=a; b.append(4); print(a)      # → [1,2,3,4] (same reference!)
```

JavaScript quirks (WellDev/Selise favorite):

```
0 == "0"        // → true (loose equality, type coercion)
0 === "0"       // → false (strict, no coercion)
typeof null     // → "object" (famous bug)
typeof NaN      // → "number"
[] + []         // → "" (empty string) ; 1 + "2" // → "12" ; "5" - 2 // → 3
```

👉 JS: `==` coerces types, `===` doesn't. Always prefer `===`.

C pointer/array basics:

```
int arr[]={10,20,30}; printf("%d", *(arr+1)); // → 20 (arr+1 = &arr[1])
char c='A'; printf("%d", c);                  // → 65 (ASCII)
```

👉 ASCII: 'A'=65, 'a'=97, '0'=48. `'a' - 'A' = 32`.

Common mistakes & tricks

- ❌ `5/2 == 2.5` in Java/C → it's **2** (integer division).
- ❌ Using `==` for Java String content → use `.equals()`.
- ❌ `i++` returns the incremented value → returns the **old** value.
- ❌ In JS, `==` and `===` are the same → `==` **coerces**.
- 👉 `typeof null = "object", typeof NaN = "number"`.

Cheat sheet

Powers of 2: $256=2^8$, $1024=2^{10}$ | 1 byte=8 bits=0.255 | hex digit=4 bits
Integer div: $5/2=2$ (Java/C), $5.0/2=2.5$ | Python $5//2=2$, $5/2=2.5$
`i++` returns old, `++i` returns new | ASCII A=65 a=97 0=48
Java String: `==` is reference, `.equals()` is value | new String → different object
JS: `==` coerces, `===` strict | `typeof null="object"`, `typeof NaN="number"`
Bitwise: $\ll n = x \times 2^n$ | $x \wedge x = 0$ | $n \& (n-1) == 0 \rightarrow$ power of 2
Static typing(Java/C++) vs Dynamic(Python/JS) | Compiled(C) vs Interpreted vs Bytecode(Java)

MCQs — attempt, then check key

Beginner (1–15) 1. In Java, `5/2` outputs: a) 2.5 b) 2 c) 3 d) error 2. Binary 1010 in decimal: a) 8 b) 10 c) 12 d) 20 3. 1 byte = how many bits? a) 4 b) 8 c) 16 d) 32 4. `i++` returns: a) incremented value b) original value c) 0 d) error 5. ASCII value of 'A': a) 60 b) 65 c) 97 d) 48 6. In Java, compare String content with: a) `==` b) `.equals()` c) `=` d) `compareTo` only 7. `2**3` in Python: a) 6 b) 8 c) 9 d) 5 8. `typeof null` in JS: a) "null" b) "object" c) "undefined" d) "number" 9. 2^{10} equals: a) 512 b) 1000 c) 1024 d) 2048 10. `5 % 2` equals: a) 2 b) 2.5 c) 1 d) 0 11. Left shift `1 << 3`: a) 3 b) 6 c) 8 d) 4 12. Python `5//2`: a) 2.5 b) 2 c) 3 d) 2.0 13. Which is dynamically typed? a) Java b) C++ c) Python d) C 14. `0 === "0"` in JS: a) true b) false c) error d) undefined 15. Hex FF in decimal: a) 15 b) 200 c) 255 d) 256

Intermediate (16–25) 16. `int a=5; int b=a++ + ++a; → b, a`: a) 12, 7 b) 11, 7 c) 12, 6 d) 10, 6 17. `String a="hi", b="hi"; a=b`: a) true b) false c) error d) null 18. `String c=new String("hi"); "hi"==c`: a) true b) false c) error d) depends 19. `0 == "0"` in JS: a) true b) false c) error d) NaN 20. `7/2*2` in Java: a) 7 b) 6 c) 7.0 d) 8 21. `typeof NaN` in JS: a) "NaN" b) "number" c) "undefined" d) "object" 22. In C, `*(arr+1)` for `arr={10,20,30}`: a) 10 b) 20 c) 30 d) address 23. `x ^ x` equals: a) x b) 0 c) 1 d) 2x 24. Java is: a) pass by reference b) pass by value (references passed by value) c) pass by name d) pass by pointer 25. `bool([])` in Python: a) True b) False c) error d) None

Difficult (26–30) 26. `n & (n-1) == 0` ($n > 0$) checks if n is: a) even b) a power of 2 c) prime d) negative 27. `1 + "2"` in JS: a) 3 b) "12" c) "3" d) error 28. `"5" - 2` in JS: a) "52" b) 3 c) "3" d) NaN 29. In Python, `a=[1,2]; b=a; b.append(3); a` is: a) [1,2] b) [1,2,3] c) error d) [3] 30. `5.0/2` in Java: a) 2 b) 2.5 c) 2.0 d) error

Answer Key — Topic 13

1-b · 2-b · 3-b · 4-b · 5-b · 6-b · 7-b · 8-b · 9-c · 10-c · 11-c · 12-b · 13-c · 14-b · 15-c · 16-a · 17-a · 18-b · 19-a · 20-b · 21-b · 22-b · 23-b · 24-b · 25-b · 26-b · 27-b · 28-b · 29-b · 30-b

Key explanations: **16** `a++` uses 5 ($a \rightarrow 6$), `++a` makes $a=7$ and uses 7 → $5+7=12$, $a=7$. **17** String literals share the pool → same reference → true. **18** `new String` creates a distinct object → `==` false. **27** number + string → string concatenation "12". **28** `-` forces numeric coercion → 3. **29** b and a reference the same list → mutation shows in a. **26** clearing the lowest set bit gives 0 only for powers of 2.

Topic 14 — Mathematics & Logical Reasoning

 **Why it matters:** Almost every BD trainee exam (Cefalo, TigerIT, DataSoft, BJIT especially) has an **aptitude section** — 8–15 questions on arithmetic, percentages, ratios, series, probability, and logic puzzles. These are fast marks with the right shortcuts and no coding needed.

Percentages, ratios, averages

- **x% of y = $(x/100) \cdot y$.** 15% of 200 = 30.
- **Percentage change = $(\text{new} - \text{old})/\text{old} \times 100$.**
- **Increase then decrease by same %:** net is a **loss** (e.g., +10% then -10% → -1%).
- **Average = sum / count.** If avg of n numbers is A, sum = n·A.
- **Ratio a:b split of total T:** parts = $T \cdot a/(a+b)$ and $T \cdot b/(a+b)$.

 **Successive % A then B → net = $A + B + (A \cdot B)/100$** (signs matter).

Speed, time, work

- **Distance = Speed × Time.** Average speed (equal distances) = $2ab/(a+b)$ (harmonic mean).
- **Work:** if A does a job in a days, A's rate = $1/a$ per day. A+B together = $1/a + 1/b$ per day → time = $ab/(a+b)$.

Number properties

- **Prime:** divisible only by 1 and itself (2 is the only even prime).
- **LCM × HCF = product of the two numbers.**
- **Divisibility:** by 3 (digit sum ÷ 3), by 9 (digit sum ÷ 9), by 4 (last two digits), by 5 (ends 0/5).
- **Even/odd:** even ± even = even, odd ± odd = even, even ± odd = odd. odd × odd = odd.

Permutations & combinations (basics)

- **Permutation** (order matters): $nPr = n!/(n-r)!$.
- **Combination** (order doesn't): $nCr = n!/(r!(n-r)!)$.
- **Factorial:** $5! = 120$. Arrangements of n distinct items = $n!$.
-  **"Arrangement/order" → permutation; "selection/group" → combination.**

Probability (basics)

- **P(event) = favorable / total**, between 0 and 1.
- **P(A or B)** (mutually exclusive) = $P(A) + P(B)$. **P(A and B)** (independent) = $P(A) \cdot P(B)$.
- **P(not A) = 1 - P(A).**
- Coin: $P(\text{head}) = 1/2$. Die: $P(\text{specific number}) = 1/6$. Two dice sum: 36 outcomes.

Series / sequences (pattern spotting)

```
2, 4, 8, 16, __ → ×2 → 32 (geometric)
1, 4, 9, 16, __ → squares → 25
2, 3, 5, 7, 11, __ → primes → 13
1, 1, 2, 3, 5, 8, __ → Fibonacci → 13
1, 8, 27, 64, __ → cubes → 125
```

🟡 Check for: +constant (arithmetic), ×constant (geometric), squares/cubes, primes, Fibonacci, alternating patterns.

Logical reasoning types

- **Syllogisms:** "All A are B; all B are C → all A are C." Watch for "some/no".
- **Blood relations:** draw a family tree; "father's sister = aunt."
- **Coding-decoding:** letter shifts (CAT→DBU is +1 each).
- **Odd one out, analogies** (A is to B as C is to ?).
- **Direction sense:** track turns (right of North = East).

Common mistakes & tricks

- ❌ "+10% then -10% returns to start" → **net -1%**.
- ❌ Confusing permutation (order) with combination (no order).
- ❌ Average speed = $(a+b)/2$ → for equal distances it's **$2ab/(a+b)$** .
- 🟡 **nCr: order doesn't matter (committee). nPr: order matters (ranking).**
- 🟡 **LCM × HCF = product of numbers.**

Cheat sheet

```
x% of y = xy/100 | %change=(new-old)/old·100 | successive: A+B+AB/100
Avg=sum/count | Ratio a:b of T → Ta/(a+b) | Dist=Speed×Time
Work together: ab/(a+b) days | Avg speed(equal dist)=2ab/(a+b)
nPr=n!/(n-r)! (order) | nCr=n!/(r!(n-r)!) (no order) | 5!=120
P=fav/total, 0..1 | P(notA)=1-P(A) | independent: P(A)·P(B)
Series: ×2 geometric, squares, cubes, primes, Fibonacci
LCM×HCF=product | 2=only even prime | div by 3 = digitsum÷3
```

MCQs — attempt, then check key

Beginner (1–15) 1. 20% of 150 = a) 20 b) 30 c) 15 d) 45 2. Average of 10, 20, 30 = a) 15 b) 20 c) 25 d) 30 3. Next: 2, 4, 8, 16, __ a) 20 b) 24 c) 32 d) 18 4. 5! = a) 25 b) 60 c) 120 d) 20 5. P(getting heads on a coin) = a) 1 b) 1/2 c) 1/4 d) 0 6. Only even prime number: a) 1 b) 2 c) 4 d) 0 7. Next: 1, 4, 9, 16, __ a) 20 b) 25 c) 24 d) 36 8. Ratio 2:3 of 100 → larger part: a) 40 b) 60 c) 50 d) 30 9. P(rolling a 4 on a die) = a) 1/4 b) 1/6 c) 1/2 d) 4/6 10. Next: 1, 1, 2, 3, 5, __ a) 7 b) 8 c) 6 d) 10 11. 15% of 200 = a) 15 b) 25 c) 30 d) 45 12. Distance if speed 60 km/h for 2 h: a) 30 b) 120 c) 60 d) 62 13. Which requires order to matter? a) combination b) permutation c) ratio d) average 14. Next: 1, 8, 27, 64, __ a) 100 b) 125 c) 81 d) 128 15. Odd one out: 2, 3, 5, 9, 11 a) 3 b) 5 c) 9 d) 11

Intermediate (16–25) 16. Price rises 10% then falls 10%. Net change: a) 0% b) –1% c) +1% d) –2% 17. nC_2 for $n=5$: a) 10 b) 20 c) 5 d) 25 18. If avg of 5 numbers is 12, their sum is: a) 12 b) 60 c) 17 d) 5 19. A does a job in 6 days, B in 3 days. Together: a) 2 days b) 4.5 days c) 9 days d) 3 days 20. $P(A \text{ and } B)$ for independent events, each $1/2$: a) 1 b) $1/2$ c) $1/4$ d) 0 21. CAT is coded DBU. Then DOG is coded: a) EPH b) CPF c) EPG d) DPH 22. $\text{LCM}(4,6) \times \text{HCF}(4,6) =$ a) 24 b) 48 c) 12 d) 10 23. Sum of two dice equals 7: number of ways: a) 5 b) 6 c) 7 d) 4 24. 12.5% as a fraction: a) $1/4$ b) $1/8$ c) $1/6$ d) $1/5$ 25. If 3 pens cost 60, 7 pens cost: a) 120 b) 140 c) 100 d) 210

Difficult (26–30) 26. Average speed for equal distances at 40 and 60 km/h: a) 50 b) 48 c) 45 d) 52 27. Arrangements of the letters in "BOOK": a) 24 b) 12 c) 6 d) 4 28. $P(\text{at least one head in 2 coin tosses})$: a) $1/4$ b) $1/2$ c) $3/4$ d) 1 29. A man's son's father is: a) grandfather b) the man himself c) uncle d) brother 30. Next in 3, 6, 11, 18, 27, __: a) 36 b) 38 c) 40 d) 34

✓ Answer Key — Topic 14

1-b · 2-b · 3-c · 4-c · 5-b · 6-b · 7-b · 8-b · 9-b · 10-b · 11-c · 12-b · 13-b · 14-b · 15-c · 16-b · 17-a · 18-b · 19-a · 20-c · 21-a · 22-a · 23-b · 24-b · 25-b · 26-b · 27-b · 28-c · 29-b · 30-b

Key explanations: **16** $+10\%$ then $-10\% \rightarrow 1.1 \times 0.9 = 0.99 \rightarrow -1\%$. **19** rates $1/6 + 1/3 = 1/2 \rightarrow 2$ days. **21** each letter $+1 \rightarrow \text{DOG} \rightarrow \text{EPH}$. **22** $\text{LCM} \times \text{HCF} = \text{product} = 4 \times 6 = 24$. **26** $2 \cdot 40 \cdot 60 / (40 + 60) = 4800 / 100 = 48$. **27** $\text{BOOK} = 4!/2! = 12$ (two O's). **28** $1 - P(\text{no heads}) = 1 - 1/4 = 3/4$. **30** differences $+3, +5, +7, +9 \rightarrow +11 \rightarrow 38$.

Topic 15 — Mock Tests

Instructions: Each mock is a realistic mixed paper at BD-trainee difficulty. **Time yourself: 50 questions in 60 minutes.** No peeking. Mark your answers, then grade with the key at the end. Aim for 35+/50 (70%). Treat wrong answers as your revision to-do list.



MOCK TEST 1 (50 questions · 60 minutes)

1. Which is NOT an OOP pillar? a) Inheritance b) Encapsulation c) Iteration d) Polymorphism
2. Overriding is resolved at: a) compile-time b) runtime c) link-time d) never
3. Array access by index is: a) $O(1)$ b) $O(n)$ c) $O(\log n)$ d) $O(n^2)$
4. Stack follows: a) FIFO b) LIFO c) priority d) random
5. Binary search complexity: a) $O(n)$ b) $O(\log n)$ c) $O(1)$ d) $O(n^2)$
6. Which SQL clause filters groups? a) WHERE b) HAVING c) ORDER BY d) SELECT
7. A primary key is: a) unique + nullable b) unique + NOT NULL c) duplicable d) always foreign
8. TCP is: a) connectionless b) reliable/connection-oriented c) fast/lossy d) a port
9. HTTPS port: a) 80 b) 443 c) 22 d) 21
10. 404 means: a) OK b) redirect c) not found d) server error
11. Which is idempotent? a) POST b) GET c) neither d) only PATCH
12. `git pull` = a) fetch only b) fetch + merge c) push d) clone
13. Singleton ensures: a) one instance b) many instances c) no instance d) two
14. Horizontal scaling means: a) bigger server b) more servers c) caching d) sharding
15. `5/2` in Java outputs: a) 2.5 b) 2 c) 3 d) error
16. Binary 1010 = a) 8 b) 10 c) 12 d) 20
17. Merge sort worst case: a) $O(n)$ b) $O(n \log n)$ c) $O(n^2)$ d) $O(2^n)$
18. Quicksort worst case: a) $O(n \log n)$ b) $O(n^2)$ c) $O(\log n)$ d) $O(n)$
19. Which normal form removes transitive dependency? a) 1NF b) 2NF c) 3NF d) BCNF
20. ACID's "I" stands for: a) Integrity b) Isolation c) Index d) Input
21. A thread differs from a process by: a) own memory b) shared memory c) no memory d) being slower
22. How many deadlock conditions must hold? a) 1 b) 2 c) 3 d) 4
23. DNS resolves: a) IP to MAC b) name to IP c) port to name d) page to server
24. Which HTTP method creates a resource? a) GET b) POST c) DELETE d) PUT
25. Inorder traversal of a BST gives: a) reverse b) sorted c) random d) level order
26. `i *= 2` loop runs in: a) $O(n)$ b) $O(\log n)$ c) $O(n^2)$ d) $O(1)$
27. Which is stable sort? a) Quick b) Selection c) Merge d) Heap
28. Encapsulation is implemented with: a) inheritance b) private fields + getters/setters c) overloading d) interfaces
29. CROSS JOIN of 3 and 4 rows gives: a) 7 b) 12 c) 0 d) 3
30. In JS, `0 == "0"` is: a) true b) false c) error d) NaN
31. Which access modifier is most restrictive (Java)? a) public b) protected c) default d) private
32. Round Robin scheduling is: a) non-preemptive b) preemptive/fair c) FIFO only d) priority-based
33. Which returns only matching rows in both tables? a) LEFT b) INNER c) FULL d) CROSS
34. A page fault occurs when: a) disk full b) page not in RAM c) CPU fails d) process ends
35. Big-O of nested loops (each n): a) $O(n)$ b) $O(n \log n)$ c) $O(n^2)$ d) $O(2n)$
36. Which is a NoSQL database? a) MySQL b) MongoDB c) Oracle d) PostgreSQL
37. 401 status code means: a) forbidden b) unauthenticated c) not found d) created
38. Which data structure for BFS? a) stack b) queue c) heap d) tree
39. Java String content comparison uses: a) `==` b) `.equals()` c) `=` d) `compareTo` only
40. Composition represents: a) is-a b) strong owns-a c) uses-a d) weak has-a
41. $\text{LCM}(4,6) \times \text{HCF}(4,6)$ = a) 24 b) 48 c) 12 d) 10
42. Which is TRUE? a) interface has constructor b) abstract class has constructor c) constructors inherited d) static overridden

43. Hash table average lookup: a) $O(1)$ b) $O(n)$ c) $O(\log n)$ d) $O(n^2)$
44. Which command stages changes in Git? a) commit b) add c) push d) status
45. `typeof null` in JS: a) "null" b) "object" c) "undefined" d) "number"
46. Which improves read performance via redundancy? a) sharding b) replication c) caching only d) normalization
47. 20% of 150 = a) 20 b) 30 c) 15 d) 45
48. Which paradigm stores subproblem results? a) greedy b) dynamic programming c) backtracking d) brute force
49. A router operates at OSI layer: a) 1 b) 2 c) 3 d) 7
50. Skewed BST search complexity: a) $O(1)$ b) $O(\log n)$ c) $O(n)$ d) $O(n^2)$

✅ Mock Test 1 — Answer Key

1-c · 2-b · 3-a · 4-b · 5-b · 6-b · 7-b · 8-b · 9-b · 10-c · 11-b · 12-b · 13-a · 14-b · 15-b · 16-b · 17-b · 18-b · 19-c · 20-b · 21-b · 22-d · 23-b · 24-b · 25-b · 26-b · 27-c · 28-b · 29-b · 30-a · 31-d · 32-b · 33-b · 34-b · 35-c · 36-b · 37-b · 38-b · 39-b · 40-b · 41-a · 42-b · 43-a · 44-b · 45-b · 46-b · 47-b · 48-b · 49-c · 50-c

Score: ___/50. Below 35? Re-read the topics behind your misses today.



MOCK TEST 2 (50 questions · 60 minutes)

1. Overloading is resolved at: a) runtime b) compile-time c) link-time d) load-time
2. Which enables multiple inheritance in Java? a) abstract class b) interfaces c) packages d) constructors
3. Linked list access time: a) $O(1)$ b) $O(\log n)$ c) $O(n)$ d) $O(n^2)$
4. Queue follows: a) LIFO b) FIFO c) random d) priority
5. Which sort is always $O(n \log n)$? a) bubble b) merge c) quick d) insertion
6. WHERE filters: a) groups b) rows c) columns d) indexes
7. Foreign key enforces: a) uniqueness b) referential integrity c) sorting d) indexing
8. UDP is best for: a) email b) file transfer c) live video d) banking
9. HTTP is: a) stateful b) stateless c) a device d) encrypted by default
10. 500 status code means: a) OK b) client error c) server error d) redirect
11. PUT method is: a) create, not idempotent b) replace, idempotent c) read-only d) delete
12. merge vs rebase: rebase creates: a) branchy history b) linear history c) a merge commit d) no history
13. Factory pattern hides: a) interface b) concrete class c) method name d) nothing
14. CAP theorem: guarantee at most: a) 1 b) 2 of 3 c) all 3 d) 4
15. `5//2` in Python: a) 2.5 b) 2 c) 3 d) 2.0
16. $2^{10} =$ a) 512 b) 1000 c) 1024 d) 2048
17. Which is divide and conquer? a) bubble sort b) merge sort c) linear search d) counting sort
18. TRUNCATE is classified as: a) DML b) DDL c) DCL d) TCL
19. 2NF removes: a) transitive dependency b) partial dependency c) atomic values d) keys
20. Virtual memory uses disk as: a) cache b) overflow (swap) c) registers d) kernel
21. Process = own memory; thread = a) own memory b) shared memory c) no memory d) kernel memory
22. SJF scheduling risk: a) convoy effect b) starvation of long jobs c) thrashing d) deadlock
23. DHCP is used to: a) resolve names b) assign IP addresses c) encrypt d) route
24. Which is safe/read-only HTTP method? a) POST b) GET c) PUT d) DELETE
25. Preorder traversal is: a) L-Root-R b) Root-L-R c) L-R-Root d) level order
26. `while(n>1) n/=2` complexity: a) $O(n)$ b) $O(\log n)$ c) $O(1)$ d) $O(n^2)$
27. Which is NOT divide and conquer? a) merge sort b) quick sort c) binary search d) bubble sort
28. Good OOP design wants: a) high coupling b) low cohesion c) high cohesion, low coupling d) high coupling, high cohesion
29. INNER JOIN of A(3) and B(4) with no matches: a) 12 b) 7 c) 0 d) 3
30. `1 + "2"` in JS: a) 3 b) "12" c) "3" d) error
31. `final` method in Java: a) can be overridden b) cannot be overridden c) cannot be called d) must be static
32. Which is preemptive scheduling? a) FCFS b) Round Robin c) basic SJF d) none
33. Which JOIN keeps all left rows? a) INNER b) LEFT c) RIGHT d) CROSS
34. Belady's anomaly affects: a) LRU b) Optimal c) FIFO d) all
35. Space complexity of in-place sort: a) $O(1)$ b) $O(n)$ c) $O(\log n)$ d) $O(n^2)$
36. Redis is a: a) relational DB b) cache/key-value store c) message queue d) load balancer
37. 403 status code means: a) unauthenticated b) authenticated but forbidden c) not found d) created
38. DFS uses: a) queue b) stack/recursion c) heap d) hash
39. `new String("hi") == "hi"` in Java: a) true b) false c) error d) null
40. Aggregation represents: a) is-a b) strong owns-a c) weak has-a d) uses-a only
41. $nC2$ for $n=5$: a) 10 b) 20 c) 5 d) 25
42. Which cannot be instantiated? a) concrete class b) abstract class c) final class d) nested class

43. Worst-case hash table lookup: a) $O(1)$ b) $O(\log n)$ c) $O(n)$ d) $O(n^2)$
44. Which downloads without merging? a) git pull b) git fetch c) git push d) git commit
45. `i++` returns: a) incremented value b) original value c) 0 d) error
46. Sharding improves: a) reads only b) write scaling c) caching d) security
47. Price +10% then -10% net: a) 0% b) -1% c) +1% d) -2%
48. Dijkstra finds: a) MST b) shortest path (non-negative) c) cycles d) sorting
49. Switch operates at OSI layer: a) 1 b) 2 c) 3 d) 7
50. Naive recursive Fibonacci is: a) $O(n)$ b) $O(n^2)$ c) $O(2^n)$ d) $O(\log n)$

✓ Mock Test 2 — Answer Key

1-b · 2-b · 3-c · 4-b · 5-b · 6-b · 7-b · 8-c · 9-b · 10-c · 11-b · 12-b · 13-b · 14-b · 15-b · 16-c · 17-b · 18-b · 19-b · 20-b · 21-b · 22-b · 23-b · 24-b · 25-b · 26-b · 27-d · 28-c · 29-c · 30-b · 31-b · 32-b · 33-b · 34-c · 35-a · 36-b · 37-b · 38-b · 39-b · 40-c · 41-a · 42-b · 43-c · 44-b · 45-b · 46-b · 47-b · 48-b · 49-b · 50-c

Score: ___/50. Compare against Mock 1 — improving scores mean the spaced repetition is working.

After both mocks: list every topic where you missed 2+ questions. Those are your Day 5 morning priorities. Re-read only those cheat sheets, not the whole book.

Topic 16 — Final Rapid Revision

Read this section the night before and the morning of the exam. It's every high-yield fact compressed. If you can rattle these off, you're ready.

The 100 must-know one-liners

OOP (1–15) 1. 4 pillars = A PIE (Abstraction, Polymorphism, Inheritance, Encapsulation). 2. Overloading = compile-time, different params. 3. Overriding = runtime, same signature. 4. Java: no multiple class inheritance (Diamond problem) → use interfaces. 5. Abstract class has constructor + state; interface doesn't. 6. Static methods are *hidden*, not overridden. 7. `final` method can't be overridden; `final` class can't be extended. 8. Encapsulation = data hiding (private + getters/setters). 9. Abstraction = detail hiding. 10. Composition = strong owns-a; Aggregation = weak has-a; Association = uses. 11. Access: private < default < protected < public. 12. Constructor: not static/final/inherited; can be overloaded. 13. Runtime polymorphism = dynamic dispatch. 14. Design goal: high cohesion, low coupling. 15. `super` = parent, `this` = current object.

Data Structures (16–30) 16. Array: $O(1)$ access, $O(n)$ insert/delete. 17. Linked list: $O(n)$ access, $O(1)$ insert@head. 18. Stack = LIFO (push/pop/peek $O(1)$). 19. Queue = FIFO (enqueue/dequeue $O(1)$). 20. Hash: $O(1)$ avg, $O(n)$ worst (collisions → chaining/open addressing). 21. BST balanced $O(\log n)$, skewed $O(n)$. 22. Heap: root=min/max, insert $O(\log n)$. 23. Inorder BST = sorted. 24. BFS = queue, DFS = stack/recursion. 25. Preorder = Root-L-R; Postorder = L-R-Root; Inorder = L-Root-R. 26. Adjacency matrix $O(V^2)$; list $O(V+E)$. 27. Priority queue = heap. 28. Binary search needs SORTED array. 29. Perfect binary tree height $h \rightarrow 2^{h+1}-1$ nodes. 30. Balanced tree height = $O(\log n)$.

Algorithms & Complexity (31–48) 31. Order: $1 < \log n < n < n \log n < n^2 < 2^n < n!$. 32. Single loop $O(n)$; nested $O(n^2)$; `i*=2` → $O(\log n)$. 33. Nested loops → multiply; sequential → add. 34. Drop constants: $3n^2+5n \rightarrow O(n^2)$. 35. Merge sort: always $O(n \log n)$, $O(n)$ space, stable. 36. Quicksort: avg $O(n \log n)$, worst $O(n^2)$, fastest in practice. 37. Heap sort: $O(n \log n)$, $O(1)$ space. 38. Insertion sort best $O(n)$ (nearly sorted). 39. Comparison sorts $\geq O(n \log n)$; counting/radix $O(n+k)$. 40. Stable sorts: Bubble, Insertion, Merge, Counting. 41. Linear search $O(n)$; binary search $O(\log n)$. 42. Naive Fibonacci $O(2^n)$; with DP $O(n)$. 43. Recursion needs a base case; space $O(\text{depth})$. 44. Greedy = one local choice; DP = store subproblems; Backtracking = try+undo. 45. BFS = shortest path (unweighted); Dijkstra = weighted non-negative; Bellman-Ford = negative edges. 46. Kruskal/Prim = MST. 47. $T(n)=2T(n/2)+O(n) \rightarrow O(n \log n)$. 48. 0/1 Knapsack = DP.

SQL & DBMS (49–65) 49. Exec order: FROM→WHERE→GROUP BY→HAVING→SELECT→ORDER BY. 50. INNER = match both; LEFT = all left; CROSS = $m \times n$. 51. WHERE filters rows; HAVING filters groups. 52. PK = unique + NOT NULL; UNIQUE allows one NULL. 53. FK → referential integrity. 54. COUNT(*) all rows; COUNT(col) skips NULLs. 55. NULL: use IS NULL (never = NULL); arithmetic with NULL = NULL. 56. DELETE (DML, WHERE, rollback) vs TRUNCATE (DDL, all, fast) vs DROP (table gone). 57. DDL: CREATE/ALTER/DROP/TRUNCATE; DML: SELECT/INSERT/UPDATE/DELETE. 58. UNION (distinct) vs UNION ALL (duplicates). 59. 1NF atomic; 2NF no partial; 3NF no transitive; BCNF determinant=key. 60. ACID: Atomicity, Consistency, Isolation, Durability. 61. Index = faster reads, slower writes. 62. Clustered index = physical order, one per table. 63. SQL = ACID/vertical; NoSQL = BASE/horizontal. 64. Self join = table joined to itself (employee-manager). 65. Denormalization = redundancy for read speed.

OS (66–78) 66. Process = own memory; thread = shared memory. 67. Deadlock needs all 4 (mutual excl, hold&wait, no preempt, circular wait). 68. Round Robin = preemptive/fair; SJF = best avg wait. 69. Page fault = page not in RAM; thrashing = constant faults. 70. Paging → internal fragmentation; segmentation → external. 71. Mutex = 1 owned; semaphore = N counter. 72. Zombie = child dead unreaped; orphan = parent dead (init adopts). 73. Context switch = pure overhead. 74. Virtual memory = private space + swap. 75. Kernel = privileged; system call = user→kernel. 76. LRU = evict least recently used; Belady's anomaly = FIFO. 77. Banker's algorithm = deadlock avoidance. 78. RR with huge quantum → behaves like FCFS.

Networks & REST & Git (79–92) 79. OSI 7 layers: "All People Seem To Need Data Processing". 80. TCP = reliable/handshake; UDP = fast/best-effort. 81. Ports: 21 FTP, 22 SSH, 25 SMTP, 53 DNS, 80 HTTP, 443 HTTPS, 3306 MySQL. 82. DNS → IP (not the page), port 53. 83. HTTP stateless; HTTPS = HTTP + TLS. 84. 2xx ok, 3xx redirect, 4xx client, 5xx server. 85. 401 unauthenticated; 403 forbidden; 404 not found. 86. Switch = L2 (MAC); Router = L3 (IP). 87. NAT = private IPs share one public IP. 88. REST = stateless, resources, HTTP methods, JSON. 89. GET/PUT/DELETE idempotent; POST not. 90. Git: add→commit(local)→push; pull = fetch+merge. 91. Never rebase shared history; revert is safe undo. 92. Merge conflict: <<<< mine == theirs >>>>.

Patterns, System Design, Code, Math (93–100) 93. Singleton = one instance (private ctor + static getInstance). 94. Factory = create without exposing concrete class; Observer = one-to-many notify. 95. SOLID = Single-resp, Open/closed, Liskov, Interface-seg, Dependency-inversion. 96. Vertical scaling = bigger box; horizontal = more boxes. 97. CAP = pick 2 of 3. 98. Java `5/2 = 2`; `.equals()` for String content; `i++` returns old value. 99. JS: `==` coerces, `===` strict; `typeof null = "object"`. 100. nPr = order matters; nCr = order doesn't; +10% then -10% = -1%.

Exam-day strategy (the meta-skills that add 5–10 marks)

- 1. Two-pass technique.** First pass: answer everything you're sure of (fast, bank the easy marks). Second pass: return to the hard ones with remaining time. Don't get stuck on Q7 while Q40 (which you know) waits.
 - 2. Elimination beats guessing.** On tough MCQs, cross out the two obviously-wrong options first. A 50/50 guess is far better than 25%. If there's **no negative marking, never leave a blank** — always guess after eliminating.
 - 3. Watch for negative marking.** If the exam penalizes wrong answers, only guess when you can eliminate at least one or two options. If unsure and can't eliminate, skip.
 - 4. Read the question fully — especially "NOT" and "EXCEPT".** Interviewers plant these to catch skimmers. "Which is NOT a pillar of OOP?" trips up people who read fast.
 - 5. Trust the first instinct** on factual recall; change an answer only if you have a concrete reason.
 - 6. Time budget.** ~1 minute per question. If a question takes >90 seconds, mark it, move on, come back.
 - 7. Output-prediction questions:** slow down and dry-run. `i++` vs `++i`, integer division, `==` vs `.equals()` — these are decided by careful reading, not speed.
 - 8. Manage the clock.** At the halfway time mark, you should be ~halfway through. If behind, speed up on the easy ones.
-

✓ Final 48-hour checklist

- ☐ Re-read all 16 cheat-sheet boxes (30 min).
 - ☐ Redo both mock tests; target 80%+.
 - ☐ Drill the ports, HTTP status codes, and sorting complexity table until automatic.
 - ☐ Practice 5 output-prediction snippets out loud.
 - ☐ Review the “frequently confused” tables (overloading/overriding, WHERE/HAVING, TCP/UDP, 401/403, merge/rebase).
 - ☐ Sleep well the night before — a rested brain recalls faster than a crammed one.
-

The three tables to memorize cold (if nothing else)

Sorting:

```
Merge  O(n log n) always, O(n) space, STABLE
Quick  O(n log n) avg / O(n²) worst, fastest in practice
Heap   O(n log n), O(1) space
Bubble/Selection/Insertion  O(n²) (insertion O(n) best)
```

HTTP status:

```
200 OK · 201 Created · 301/302 Redirect · 400 Bad Request
401 Unauthenticated · 403 Forbidden · 404 Not Found · 500 Server Error
4xx = client's fault · 5xx = server's fault
```

DS operations:

```
Array: access O(1), insert/delete O(n)
LinkedList: access O(n), insert@head O(1)
Hash: O(1) avg, O(n) worst
BST: O(log n) balanced, O(n) skewed
```

You've got everything you need. This handbook covers the ~20% of concepts behind ~80% of the questions on the Aug 1 WellDev exam. Do the mocks, drill the cheat sheets, and walk in confident.

Good luck — go bank those marks! 🚀